



SESSION 10

BUILDING WEBSITES USING NODE.JS

Mục tiêu học tập

Trong buổi học này, học sinh sẽ học cách:

- Giải thích quá trình tạo ứng dụng Web bằng Node.js

- Mô tả các bước để kết nối ứng dụng Web với đám mây MongoDB cơ sở dữ liệu

- Liệt kê các bước để tải ứng dụng Web lên kho lưu trữ GitHub

- Giải thích quy trình triển khai ứng dụng Web trên Render từ kho lưu trữ GitHub

Node.js có thể được sử dụng để tạo một trang web hoàn chỉnh. Ứng dụng web front-end có thể được kết nối với cơ sở dữ liệu back-end. GitHub và Render đóng vai trò quan trọng trong quá trình triển khai web bằng Node.js.

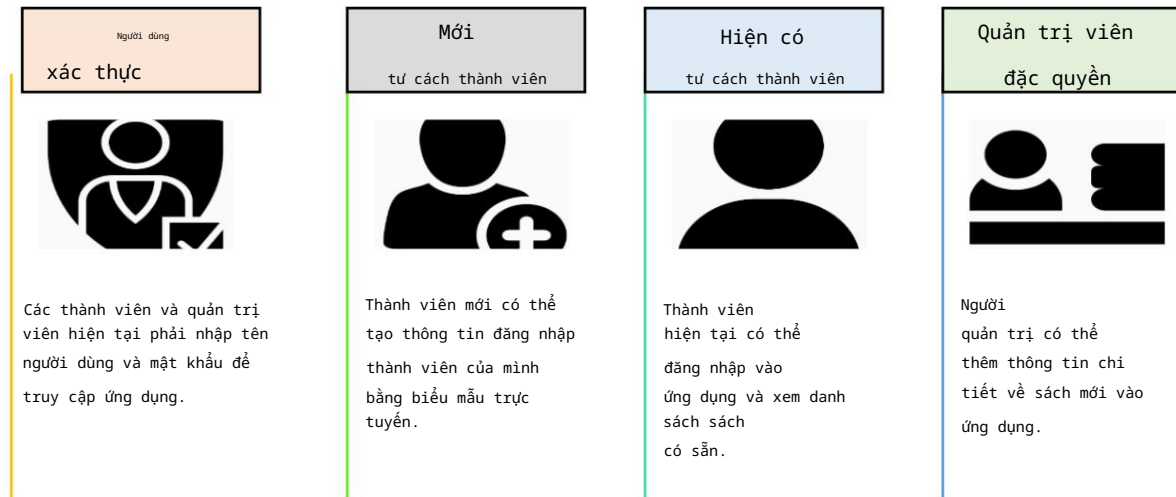
Phiên này bắt đầu bằng việc giải thích quy trình truy xuất chuỗi kết nối của cơ sở dữ liệu đám mây MongoDB. Sau đó, nó sẽ mô tả quy trình tạo ứng dụng web bằng Node.js.

Phiên này giải thích quy trình tải ứng dụng web lên kho lưu trữ GitHub. Nó cũng trình bày chi tiết các bước triển khai ứng dụng trên Render từ kho lưu trữ GitHub. Cuối cùng, phiên này kết thúc bằng cách minh họa việc kiểm tra chức năng của ứng dụng trên trình duyệt web.

10.1 Tổng quan

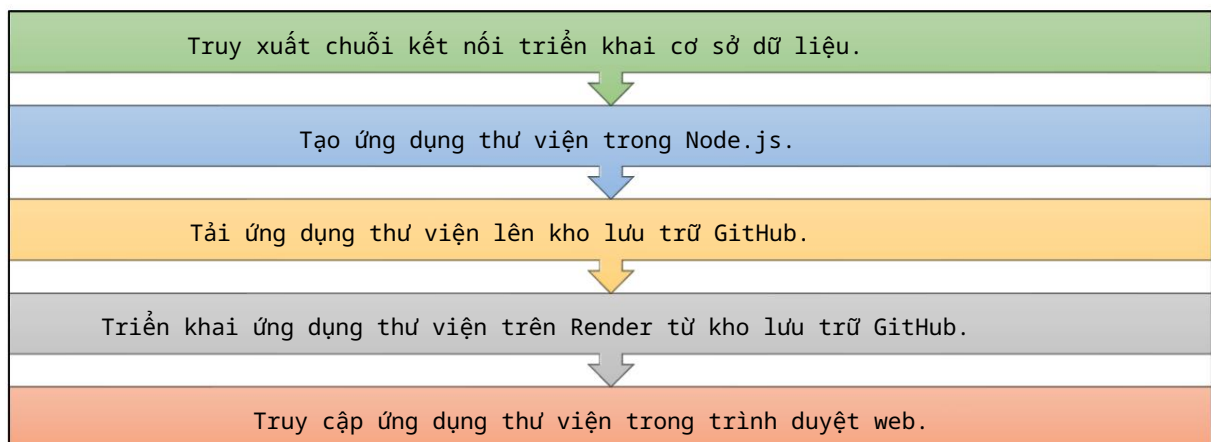
Node.js có thể được sử dụng để tạo cả ứng dụng front-end và back-end. Trong phần này, một ứng dụng thư viện đơn giản được tạo bằng Node.js và

Cơ sở dữ liệu đám mây MongoDB có tên là Thư viện. Hình 10.1 thể hiện các đặc điểm của ứng dụng thư viện.



Hình 10.1: Ứng dụng thư viện

Các bước tạo ứng dụng thư viện như sau:

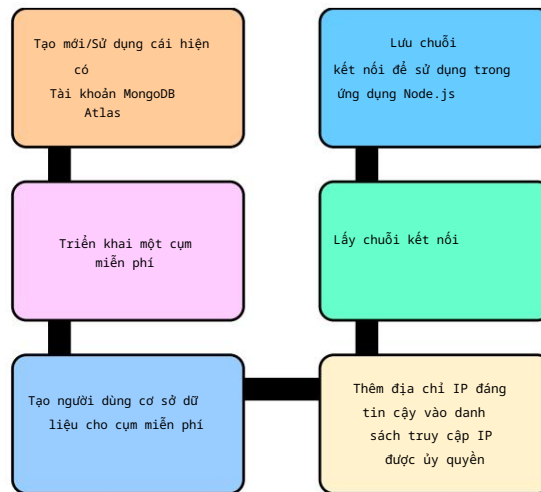


10.2 Lấy chuỗi kết nối từ MongoDB Cloud

Điều kiện tiên quyết để tạo ứng dụng thư viện là triển khai cơ sở dữ liệu trên nền tảng đám mây MongoDB và truy xuất chuỗi kết nối triển khai cơ sở dữ liệu.

Chuỗi kết nối sau đó được sử dụng để kết nối ứng dụng Thư viện với cơ sở dữ liệu đám mây để lưu trữ, truy xuất và xử lý dữ liệu.

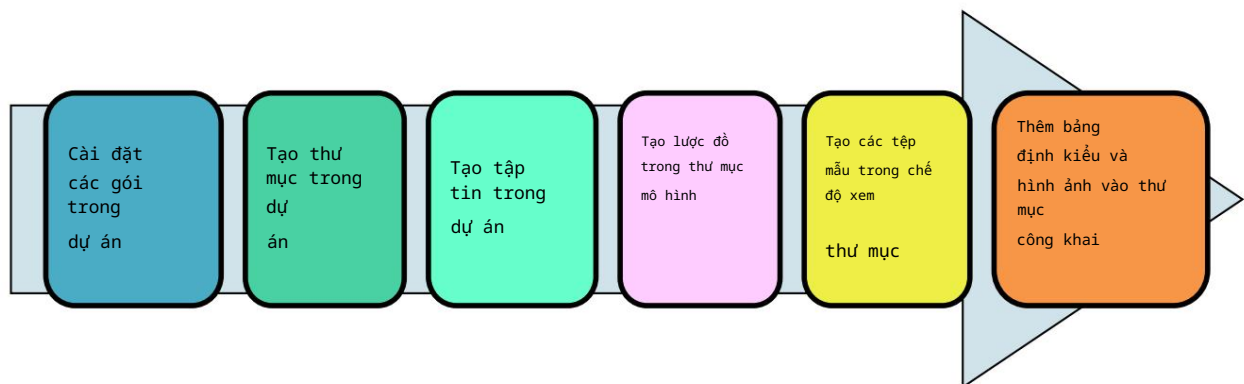
Sau đây là các bước cần thực hiện để hoàn thành nhiệm vụ:



Tham khảo Phụ lục A để biết mô tả chi tiết về cách lấy chuỗi kết nối từ đám mây MongoDB.

10.3 Tạo ứng dụng thư viện trong Node.js

Bước tiếp theo là tạo ứng dụng thư viện trong Node.js bao gồm các hành động sau:



10.3.1 Cài đặt các gói trong dự án

Để cài đặt các gói Node.js cần thiết, hãy thực hiện các tác vụ sau:

1. Tạo thư mục cho ứng dụng. Để thực hiện:
 - a. Mở Dấu nhắc lệnh. b. Điều hướng đến ổ đĩa cục bộ nơi Node.js được cài đặt. c. Điều hướng đến thư mục để tạo thư mục ứng dụng.
 - d. Tạo một thư mục có tên `library_application` và thay đổi thành thư mục đó.

Hình 10.2 hiển thị đường dẫn thư mục.

```
C:\Users\Linda\nodejs>cd library_application
```

Hình 10.2: Tạo thư mục mới

2. Tạo tệp package.json để khởi tạo dự án Node.js mới cho ứng dụng. Nhập lệnh tại dấu nhắc và nhấn Enter.

```
npm init -y
```

Lệnh này tạo tệp package.json với các giá trị mặc định trong thư mục gốc của ứng dụng, library_application. Hình 10.3 hiển thị kết quả đầu ra của lệnh.

```
C:\Users\Linda\nodejs\library_application>npm init -y
Wrote to C:\Users\Linda\nodejs\library_application\package.json:

{
  "name": "library_application",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Hình 10.3: Tạo một dự án Node.js mới

Thông tin trong Hình 10.4 được lưu trữ trong tệp package.json như được chỉ định trong hình ảnh.

3. Cài đặt các gói cần thiết vào thư mục của ứng dụng.

Bảng 10.1 liệt kê mục đích của bảy gói phải được cài đặt trong thư mục ứng dụng.

Gói hộ chiếu	Sự miêu tả
	Passport là một phần mềm trung gian trong Node.js cung cấp các chiến lược xác thực bằng cách sử dụng tên người dùng-mật khẩu, Twitter, Facebook, Instagram và sorm
thể hiện	Một khuôn khổ ứng dụng web tạo điều kiện cho việc phát triển ứng dụng nhanh chóng
ejs	Một thư viện JavaScript dành cho JavaScript nhúng (EJS) Mẫu được sử dụng để tạo siêu văn bản Mẫu Ngôn ngữ đánh dấu (HTML)

Gói cây mangut	Sự miêu tả
	Thư viện Mô hình hóa dữ liệu đối tượng (ODM) dành cho MongoDB được sử dụng để tạo điều kiện giao tiếp với cơ sở dữ liệu MongoDB và thực hiện các hoạt động cơ sở dữ liệu
trình phân tích cú pháp cơ thể	Một phần mềm trung gian Node.js được sử dụng để phân tích cú pháp nội dung của các yêu cầu HTTP POST
phiên hợp nhanh	Một phần mềm trung gian phiên để quản lý dữ liệu phiên
hệ chiếu địa phương	Một gói được sử dụng để thực hiện quy trình xác thực người dùng
passport-local-mongoose	Một plug-in mongoose giúp người dùng xác thực và quản lý phiên

Bảng 10.1: Các gói sẽ được thêm vào Dự án

Tất cả các gói này có thể được cài đặt chỉ bằng một lệnh. Để cài đặt các gói, hãy nhập lệnh tại dấu nhắc và nhấn Enter:

```
npm install passport express ejs mongoose body-parser express-session passport-local passport-local-mongoose
```

Hình 10.4 hiển thị kết quả của lệnh.

```
C:\Users\Linda\nodejs\library_application>npm install express ejs
mongoose body-parser express-session passport-local passport-local-mongoose

added 112 packages, and audited 113 packages in 2s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Hình 10.4: Cài đặt các gói

Sau khi cài đặt các gói, hãy quan sát cấu trúc của thư mục `library_application` như thể hiện trong Hình 10.5.

```
C:\Users\Linda\nodejs\library_application>dir
Volume in drive C is Windows
Volume Serial Number is 66C5-B39B

Directory of C:\Users\Linda\nodejs\library_application

11/30/2023  06:56 PM    <DIR>          .
11/30/2023  06:36 PM    <DIR>          ..
11/30/2023  06:58 PM    <DIR>          node_modules
11/30/2023  06:58 PM                43,889 package-lock.json
11/30/2023  06:58 PM                467 package.json
                2 File(s)              44,356 bytes
                3 Dir(s)  275,915,255,808 bytes free
```

Hình 10.5: Danh sách thư mục `library_application`

Thư mục `node_modules` và tệp `package-lock.json` được tự động tạo khi lệnh `npm` được thực thi để cài đặt các gói khác nhau trong dự án. Tệp `package-lock.json` bao gồm thông tin chi tiết đầy đủ về tất cả các gói đã cài đặt trong dự án. Không nên chỉnh sửa thủ công tệp này. Ngoài ra, tệp `package.json` được tự động cập nhật với số phiên bản của các gói đã cài đặt như thể hiện trong Hình 10.6.



```
{
  "name": "library_application",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "body-parser": "^1.20.2",
    "ejs": "^3.1.9",
    "express": "^4.18.2",
    "express-session": "^1.17.3",
    "mongoose": "^8.0.2",
    "passport-local": "^1.0.0",
    "passport-local-mongoose": "^8.0.0"
  }
}
```

Hình 10.6: Tệp `package.json` có số phiên bản

- Mở tệp `package.json` trong VS Code. Chỉnh sửa tệp để cập nhật giá trị của trường `chính` và bao gồm các trường `engine` và `node` như được hiển thị trong Đoạn mã 1.

Đoạn mã 1:

```
{
  "tên": "library_app",
  "phiên bản": "1.0.0",
```

```

"description": "", "main":
"app.js", "scripts": { "test":
"echo \"Lỗi: không chỉ
định bài kiểm tra\" && thoát 1" }, "keywords": [], "author": "", "license": "ISC", "dependencies":

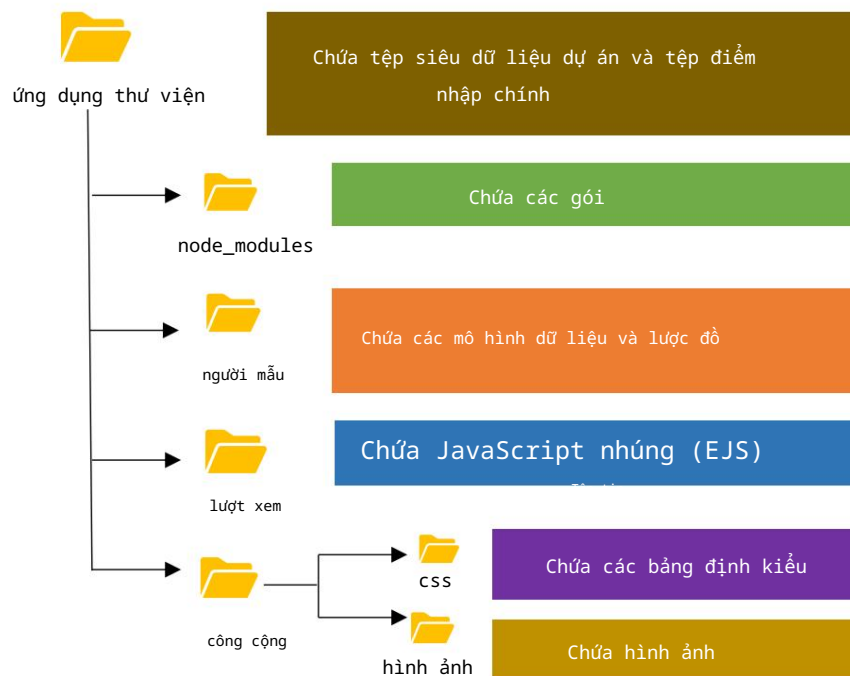
{ "body-parser": "^1.20.2",
"ejs": "^3.1.9",
"express": "^4.18.2", "express-
session": "^1.17.3", "mongoose":
"^8.0.0", "passport": "^0.6.0", "passport-local":
"^1.0.0", "passport-local-
mongoose": "^8.0.0" }, "engines":
{ "node": "20.7.0" } }

```

Lưu và đóng tệp package.json đã cập nhật.

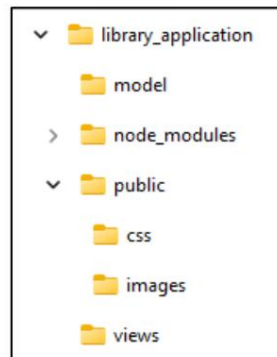
10.3.2 Tạo thư mục trong dự án

Để tổ chức cấu trúc dự án, hãy tạo các thư mục và thư mục con cần thiết như thể hiện trong Hình 10.7.



Hình 10.7: Cấu trúc thư mục dự án

Hình 10.8 hiển thị danh sách thư mục library_application.



Hình 10.8: Danh sách thư mục library_application

10.3.3 Tạo tệp trong dự án

Để xác định thiết kế giao diện người dùng, logic và chức năng của dự án, hãy tạo các tệp mã nguồn cần thiết trong các thư mục dự án đã chỉ định. • Tạo lược đồ trong thư mục mô hình

Để xác định lược đồ cho các bộ sưu tập cơ sở dữ liệu Thư viện, hãy tạo hai mô hình, User.js và Book.js, trong thư mục mô hình. Các lược đồ này thể hiện cấu trúc của các bộ sưu tập sách và người dùng sẽ được lưu trữ trong

Cơ sở dữ liệu thư viện. Mô hình cho phép các nhà phát triển tạo, cập nhật và truy xuất dữ liệu trong ứng dụng.

1. Tạo User.js

Đoạn mã 2 hiển thị mã cho User.js. Lưu mã với tên tệp là User.js trong thư mục mô hình.

Đoạn mã 2:

```
const mongoose = require('mongoose') const Schema
= mongoose.Schema const passportLocalMongoose
= require('passport-local-mongoose'); var User = new Schema({ username:
{ type:
String }, password: { type:
String } })
```

```
Người dùng.plugin(passportLocalMongoose);
module.exports = mongoose.model('Người dùng', Người dùng)
```


Các thao tác được trình bày trong tệp User.js như sau:

- Gói mongoose được nhập và sử dụng để tương tác với cơ sở dữ liệu MongoDB.
- Câu lệnh `const Schema = mongoose.Schema` nhập lớp Schema từ gói mongoose. Lớp Schema được sử dụng để định nghĩa cấu trúc của các bộ sưu tập sẽ được lưu trữ trong cơ sở dữ liệu.
- Câu lệnh `var User = new Schema({ })` định nghĩa lược đồ cho mô hình Người dùng. Lược đồ này chỉ định hai trường, tên người dùng và mật khẩu, cùng với kiểu dữ liệu của chúng.
- Plugin `passportLocalMongoose` được thêm vào lược đồ Người dùng cho mục đích xác thực.
- `mongoose.model('User', User)` tạo một đối tượng mô hình có tên là User dựa trên lược đồ User. Mongoose tạo một bộ sưu tập dựa trên mô hình User. Nó tự động gán tên của bộ sưu tập là `users`, là phiên bản số nhiều, viết thường của tên mô hình. Câu lệnh `module.exports` xuất User

mô hình dưới dạng một mô-đun. Việc xuất đối tượng mô hình cho phép các nhà phát triển truy cập mô hình Người dùng trong bất kỳ tệp hoặc dự án nào khác.

2. Tạo Book.js

Đoạn mã 3 tạo một mô hình Sách để lưu trữ thông tin chi tiết về sách trong cơ sở dữ liệu Thư viện. Đoạn mã này tạo một đối tượng Schema có tên là `bookSchema`. Sau đó, nó tạo ra một mô hình có tên là `Book` dựa trên `bookSchema`. Vì các mô hình đại diện cho các bộ sưu tập, Mongoose tạo ra một bộ sưu tập dựa trên `Book`. mô hình trong cơ sở dữ liệu Thư viện và tự động gán tên bộ sưu tập là `sách`. Mã xuất mô hình Sách để có thể truy cập được trong tệp hoặc dự án khác.

Lưu mã với tên tệp là `Book.js` trong thư mục mô hình.

Đoạn mã 3:

```
const mongoose = require("mongoose");
const bookSchema = new mongoose.Schema({
  Book_id: Số,
  Book_name: Chuỗi,
  Tên tác giả: Chuỗi,
  Giá: Chuỗi, // Giả sử giá là một số
  Nhóm tuổi: Chuỗi,
  Book_type: Chuỗi,
});
const Book = mongoose.model("Sách", bookSchema);
module.exports = Sách;
```

Sau khi tạo các tệp lược đồ, cấu trúc của thư mục mô hình sẽ xuất hiện như trong Hình 10.9.

Name	Date modified	Type	Size
 Book	08-11-2023 20:34	JS File	1 KB
 User	08-11-2023 20:11	JS File	1 KB

Hình 10.9: Danh sách Thư mục mô hình

- Tạo các tệp mẫu trong thư mục chế độ xem

Để hiển thị các trang HTML cho ứng dụng, hãy tạo các tệp mẫu hoặc .ejs các tệp trong thư mục views. Các tệp .ejs được sử dụng để cung cấp mã đánh dấu HTML bằng cách nhúng JavaScript cần thiết vào tệp ứng dụng Node.js chính, app.js.

1. Tạo register.ejs

Đoạn mã 4 minh họa mã HTML cho biểu mẫu đăng ký dành cho thành viên thư viện mới. Biểu mẫu đã hoàn thành, sau khi được gửi, sẽ được chuyển đến điểm cuối /register để xử lý tiếp. Lưu mã với tên tệp register.ejs trong thư mục views.

Đoạn mã 4:

```
<!DOCTYPEhtml>
<html>
<đầu>
  <link rel="stylesheet" type="text/css" href="/css/
register.css">
  <title>Đăng ký làm thành viên thư viện</title>
</đầu>
<thân>
  <div class="container">
    
    <h1>Mẫu đăng ký làm thành viên thư viện</h1>
    <form action="/register" method="POST" class="form-
bảng">
      <bảng>
        <tr>
          <td><label for="username">Tên người dùng</label></td>
          <td><input type="text" name="username" id="username" placeholder="Tên người
dùng" autocomplete="off"></td>
        </tr>
        <tr>
          <td><label for="password">Mật khẩu</label></td>
```

```

    <td><input type="password" name="password" id="password" placeholder="Mật
khẩu"></td> </tr>

    <tr>
    <td></td>
    <td><button>Đăng ký</button></td> </tr> </table> </form>
<p><a
href="/"
logout">Trang
        chủ</a></p> </div> </body> </html>

```

2. Tạo login.ejs

Đoạn mã 5 minh họa mã HTML cho biểu mẫu đăng nhập của ứng dụng. Biểu mẫu đã hoàn thành, sau khi được gửi, sẽ được chuyển đến điểm cuối /login để xử lý thêm. Lưu mã với tên tệp là login.ejs trong thư mục views.

Đoạn mã 5:

```

<!DOCTYPE html> <html>

<đầu>
    <link rel="stylesheet" type="text/css" href="/css/register.css">

    <title>Đăng nhập thành viên</title> </head>
<body> <div
class="container">
     <h1>Đăng nhập thành viên</
h1> <form action="/login"
method="POST">
    <bảng>
    <tr>
    <td><label for="username">Tên người dùng</label></td> <td><input type="text"
name="username" id="username" placeholder="Tên người dùng" autocomplete="off"></td>

    </tr>
    <tr>
    <td><label for="password">Mật khẩu</label></td> <td><input type="password" name="password" id="password"
placeholder="Mật khẩu"></td> </tr>

    <tr>
    <td></td>
    <td><button>Đăng nhập</button></td> </tr>

```

```
</table> </
form> <p><a
    href="/logout">Trang chủ</a></p> </div> </body> </html>
```

3. Tạo home.ejs

Đoạn mã 6 minh họa mã HTML cho trang chủ của ứng dụng. Trên trang chủ của ứng dụng, người dùng có thể:

- Đăng ký làm thành viên mới • Đăng nhập với tư cách là thành viên hiện tại
- Đăng nhập với tư cách là quản trị viên

Lưu mã với tên tệp là home.ejs trong thư mục views.

Đoạn mã 6:

```
<!DOCTYPE html> <html>

<đầu>
    <link rel="stylesheet" type="text/css" href="/css/home.css">
<title>Đăng nhập thành viên</title> </
    head> <body> <div class="container"> <div
class="right">
<h1>Trang
chủ thư viện</h1> <table align="center"
style="width:50%"> <tr>

    <td align="center"><a href="/register">Đăng ký
Thành viên Người dùng</a></
    td> </tr>
<tr>
    <td align="center"><a href="/login">Thành viên
Đăng nhập</a></td> </
    tr>
<tr>
    <td align="center"><a href="/admin">Quản trị viên
Đăng nhập</a></td> </
    tr> </
table> </
div> <div
class="left">
     </div>
```

```

</div> </
body> </
html>

```

4. Tạo admin-login.ejs

Đoạn mã 7 minh họa mã HTML cho trang đăng nhập của quản trị viên. Quản trị viên có thể nhập tên người dùng và mật khẩu vào biểu mẫu đăng nhập. Biểu mẫu đã hoàn thành, sau khi được gửi, sẽ được gửi đến điểm cuối /admin-login để xử lý thêm.

Lưu mã với tên tệp là admin-login.ejs trong thư mục views.

Đoạn mã 7:

```

<!DOCTYPEhtml>
<html>
<head>
<link rel="stylesheet" type="text/css" href="/css/register.css">
<title>Đăng nhập quản trị</title> </head>
  <body> <div class="container">

     <h1>Đăng nhập quản trị</h1>
    <form action="/admin-login"
method="post"> <table>

  <tr>
  <td><label for="username">Tên người dùng</label></td> <td><input type="text"
name="username" id="username"
placeholder="Tên người dùng" autocomplete="tắt"></td>
  </tr>
  <tr>
  <td><label for="password">Mật khẩu</label></td> <td><input type="password"
name="password"
id="password" placeholder="Mật khẩu"></td> </tr>

  <tr>
  <td></td>
  <td><button>Đăng nhập</button></td> </tr> </table> </
form>

<p><a href="/logout">Trang chủ</a></p>
  </div> </
body> </
html>

```

5. Tạo admin-dashboard.ejs

Đoạn mã 8 minh họa mã HTML cho trang bảng điều khiển quản trị. Trên trang này, quản trị viên có thể thêm thông tin chi tiết về sách mới vào bộ sưu tập sách của thư viện. Khi nhấp vào nút Thêm Sách, biểu mẫu đã hoàn thành sẽ được gửi đến điểm cuối `/admin-dashboard/add-book` để xử lý thêm.

Lưu mã với tên tệp là `admin-dashboard.ejs` trong chế độ xem thư mục.

Đoạn mã 8:

```
<!DOCTYPEhtml>
<html>
<đầu>
  <link rel="stylesheet" type="text/css" href="/css/
admindash.css">
  <title>Bảng điều khiển quản trị</title>
</đầu>
<thân>
  <div class="container">
    <form action="/admin-dashboard/add-book" method="post">

    <bảng> <tr>
      <td></td>
      <td><h1>Tạo chi tiết sách</h1></td>
    </tr>
    <tr>
      <td><label for="Book_id">Mã sách:</label></td>
      <td><input type="number" name="Book_id" required autocomplete="off"></
td>
    </tr>
    <tr>
      <td><label for="Book_name">Tên sách:</label></td>
      <td><input type="text" name="Book_name" required autocomplete="off"></
td>
    </tr>
    <tr>
      <td><label for="Author_name">Tác giả
Tên:</label></td>
      <td><input type="text" name="Author_name" required autocomplete="off"></
td>
    </tr>
    <tr>
      <td><label for="Price">Giá:</label></td>
      <td><input type="text" name="Price" required
autocomplete="off"></td>
    </tr>
```

```

<tr>

<td><label for="Age_group">Nhóm tuổi:</label></td> <td><input type="text"
name="Age_group" required autocomplete="off"></td> </tr>

<tr>
<td><label for="Book_type">Loại sách:</label></td> <td><input type="text"
name="Book_type" required autocomplete="off"></td> </tr> </table> <div
class="center-button-container">
<button
class="center-
button" type="submit">Thêm

Sách</button> </div>
</form> </
div> <p><a
href="/"
logout">Đăng xuất</a></p> </body> </html>

```

6. Tạo booklist.ejs

Đoạn mã 9 minh họa mã HTML của chế độ xem hiển thị danh sách sách được lấy từ bộ sưu tập sách.

Việc sử dụng JavaScript nhúng (<% %>) cho thấy rằng: • Dữ liệu được truy xuất và hiển thị động bởi phía máy chủ ứng dụng. • Dữ liệu sách được đưa vào mẫu HTML khi chạy.

Lưu mã với tên tệp là booklist.ejs trong thư mục views.

Đoạn mã 9:

```

<!DOCTYPEhtml>
<html>
<đầu>
    <link rel="stylesheet" type="text/css"
href="/css/bookstyle.css"> <title>Danh
sách sách</title> </head> <body> <div

class="content"> <div
class="container"> <div
class="column1"> <h1>Danh sách
sách</h1> <table class="center">

<tr>

```

```

<th>Mã sách</th> <th>Sách</th>
<th>Tác giả</th> <th>Giá</th>
<th>Độ tuổi</th> <th>Loại sách</th>
</tr> <%
books.forEach(function(book) { %> <tr>

    <td><%= book.Book_id %></td> <td><%=
book.Book_name %></td> <td><%= book.Author_name
%></td> <td><%= book.Price %></td> <td> <%=
book.Age_group %></td> <td> <%=
book.Book_type %></td> </tr> <% }); %> </table> </
div> </div> </div> <p><a href="/logout">Đăng xuất</
a></p>
</body> </html>

```

7. Tạo error.ejs

Đoạn mã 10 minh họa mã HTML cho trang lỗi khi đăng nhập thành viên.

Trang lỗi: • Hiện

thị thông báo lỗi cho thành viên. • Cung cấp tùy chọn
để quay lại trang đăng nhập.

Lưu mã với tên tệp là error.ejs trong thư mục views.

Đoạn mã 10:

```

<!-- error.ejs --> <!DOCTYPE
html>
<html>
<đầu>

    <link rel="stylesheet" type="text/css" href="/css/error.css"> <title>Lỗi</
title> </head> <body> <div
    class="container"> <h1>Lỗi</h1> <p><%=
errorMessage
%></p>

```



```
<p><a href="/login">Quay lại đăng nhập</a></p>

</div> </
body> </
html>
```

8. Tạo admin-error.ejs

Đoạn mã 11 minh họa mã HTML cho trang lỗi đăng nhập quản trị viên. Trang lỗi: • [Hiển thị thông báo lỗi cho quản trị viên](#) • [Cung cấp tùy chọn quay lại trang đăng nhập quản trị viên](#)

Lưu mã với tên tệp là admin-error.ejs trong thư mục views.

Đoạn mã 11:

```
<!-- error.ejs --> <!DOCTYPE
html> <html>

<đầu>
<link rel="stylesheet" type="text/css" href="/css/error.css"> <title>Lỗi</
title> </head> <body> <div class="container">


<h1>Lỗi</h1> <p><%=
errorMessage %></p> <p><a href="/
admin">Quay lại trang đăng nhập quản trị</a></p> </div> </body> </html>
```

Sau khi tạo các tệp .ejs, hãy tìm cấu trúc của thư mục views như thể hiện trong Hình 10.10.

Name	Date modified	Type	Size
 admin-dashboard	08-11-2023 21:47	EJS File	2 KB
 admin-error	08-11-2023 22:05	EJS File	1 KB
 admin-login	08-11-2023 21:43	EJS File	1 KB
 booklist	08-11-2023 21:53	EJS File	1 KB
 error	08-11-2023 22:00	EJS File	1 KB
 home	08-11-2023 22:07	EJS File	1 KB
 login	08-11-2023 21:39	EJS File	2 KB
 register	08-11-2023 21:40	EJS File	2 KB

Hình 10.10: Danh sách các chế độ xem Thư mục

- Thêm Stylesheets và Hình ảnh vào Thư mục công cộng

Tiếp theo, hãy bao gồm các tệp .css và hình ảnh cần thiết vào các thư mục public\css và public\images tương ứng.

1. Tạo register.css

Đoạn mã 12 hiển thị bảng định dạng cho biểu mẫu đăng ký thành viên mới. Lưu mã với tên tệp là register.css trong thư mục public\css.

Đoạn mã 12:

```
/* register.css */ body { margin:
0; padding:
0; font-family:
Arial, sans-serif;
background-color: lightblue; /* Màu nền cho toàn bộ
trang */ } h1 { border: 2px #eee solid; color: brown; text-align: center; padding: 10px;
margin: 0; /* Thêm dòng này để
xóa
bất kỳ lề mặc định nào */ } .container
{ display: flex; flex-
direction: column; /* Thay đổi
thành bố cục cột */
justify-content: center;
```

```
align-items: center; height: 100vh;
text-align: center; /*
Căn giữa nội dung */ } .form-table { background-color: #fff; /* Màu nền cho

biểu mẫu chứa */ đệm: 20px;
bán kính đường viền:
8px; bóng hộp: 0 4px 6px rgba(0,
0, 0, 1); } bảng { thu gọn đường viền: thu gọn; chiều rộng: 100%; }

bảng td { đệm:
10px; }

đầu vào
{ chiều rộng: 100%;
đệm: 10px; lề: 5px 0;
đường viền: 1px solid
#ccc; bán kính đường viền: 4px; }

nút { chiều
rộng: 100%; đệm:
10px; màu nền: #007BFF; /
* Màu nền của nút */ màu: #fff; /* Màu văn bản của nút */ đường viền: không có; bán kính đường
viền: 4px; con trỏ: con trỏ; }

button:hover { background-
color: #0056b3; /* Màu nền của nút khi di chuột qua */ }
```

2. Tạo home.css

Đoạn mã 13 hiển thị bảng định kiểu cho trang chủ của ứng dụng.

Lưu mã với tên tệp là home.css trong thư mục public\css.

Đoạn mã 13:

```
bảng, th, td { đường
viền: 1px liền khối; cỡ chữ:
30px; }

thân { màu
nền: xanh nhạt; } h1 { đường viền: 2px #eee solid;

màu:
nâu; căn chỉnh văn bản: ở giữa; đệm:
10px; } .center-image
{ hiển thị: khối; lề: 0 tự động;
chiều rộng tối đa: 100%;

chiều cao: tự động; }

.container { chiều
cao: 75%; tràn:
ẩn; } .right { chiều rộng:
800px; float:
phải; lề trên cùng:
100px; } .left { lề
trên cùng: 125px; chiều rộng:

tự động;
tràn: ẩn; }
```

3. Tạo error.css

Đoạn mã 14 hiển thị bảng định kiểu cho các trang lỗi của ứng dụng.
Lưu mã với tên tệp là error.css trong thư mục public\css.

Đoạn mã 14:

```
.container
{ display: flex; flex-
direction: column; align-items: center;
width: 100%; /* Đặt chiều rộng
của container thành 100% */
```

```
max-width: 1000px; /* Tùy chọn, đặt chiều rộng tối đa cho vùng chứa */ margin: 0 auto; /*  
Căn giữa vùng chứa theo chiều  
ngang */ } body { margin: 0; padding: 0; font-family: Arial, sans-serif; background-  
  
color:  
lightblue;  
display: flex;  
flex-direction: column; align-items: center; justify-  
content: flex-start; min-height: 100vh; }
```

4. Tạo bookstyle.css

Đoạn mã 15 hiển thị bảng định dạng để hiển thị chi tiết sách. Lưu mã với tên tệp là bookstyle.css trong thư mục public\css.

Đoạn mã 15:

```
container { hiển  
thị: lưới; cột mẫu lưới:  
100px 150px 200px; căn chỉnh văn bản: giữa; } .column1 { float: trái;  
chiều rộng: 90%; đệm: 10px; chiều  
  
cao: tự động;  
màu nền: xanh nhạt; }  
body { màu nền:  
xám; } .content { chiều  
rộng: 100%; vị trí:  
tuyệt đối; trên cùng: 10%; } .center { hiển thị:  
  
bảng;  
chiều cao: 100%; chiều rộng: 100%; } h2 {
```

```
viền: 2px #eee solid; màu: Đỏ; căn chỉnh  
văn bản: giữa;  
đệm: 10px; }
```

5. Tạo admin-dash.css

Đoạn mã 16 hiển thị bảng định kiểu cho bảng điều khiển quản trị viên. Lưu mã với tên tệp admin-dash.css trong thư mục public\css.

Đoạn mã 16:

```
/* admin-dash.css */ body { margin: 0;  
padding:  
  0; font-family:  
  Arial, sans-serif;  
  background-color: lightblue; display: flex; flex-  
  direction: column; align-items: center; justify-  
  content: center; /* Căn  
  giữa cả hai theo chiều ngang  
  
và theo chiều dọc */  
  chiều cao tối thiểu: 100vh; }  
  
.container { hiển  
  thị: flex; hướng flex:  
  cột; căn chỉnh các mục: giữa; chiều  
  rộng: 100%; chiều rộng tối đa:  
  1000px; lề: 0 tự  
  động; }  
  
h1  
  { đường viền: 2px #eee rắn; màu: nâu;  
  căn chỉnh văn bản:  
  giữa; đệm: 10px; lề: 0; }  
  
hình thức  
  { chiều rộng: 60%; }  
  
bảng  
  { border-collapse: thu gọn;
```

```
chiều rộng: 100%;  
lề: 20px tự động;  
}  
  
bảng td {  
đệm: 10px;  
}  
  
nhấn {  
font-weight: đậm;  
hiển thị: khối;  
căn chỉnh văn bản: trái;  
}  
  
đầu vào {  
chiều rộng: 100%;  
đệm: 10px;  
lề: 5px 0;  
đường viền: 1px solid #ccc;  
bán kính đường viền: 4px;  
}  
  
.center-button-container {  
căn chỉnh văn bản: giữa; /* Căn giữa theo chiều ngang */  
margin-top: 20px; /* Điều chỉnh lề trên theo yêu cầu */  
  
}  
  
.nút-trung-tâm {  
chiều rộng: 150px; /* Điều chỉnh chiều rộng theo yêu cầu */  
chiều cao: 50px; /* Điều chỉnh chiều cao theo yêu cầu */  
màu nền: #007BFF; /* Màu nền xanh */  
  
màu sắc: #fff;  
đường viền: không có;  
con trỏ: con trỏ;  
}
```

6. Thêm hình ảnh

Tải xuống từ https://commons.wikimedia.org/wiki/File:Books_HD_%288314929977%29_sach1.jpeg

.jpg và tải nó lên thư mục public\images.

Sau khi tạo các tệp .css, hãy tìm cấu trúc của public\css thư mục như thể hiện trong Hình 10.11.

Name	Date modified	Type	Size
 admindash	09-11-2023 14:48	Cascading Sty...	2 KB
 bookstyle	09-11-2023 14:36	Cascading Sty...	1 KB
 error	09-11-2023 14:33	Cascading Sty...	1 KB
 home	09-11-2023 14:53	Cascading Sty...	1 KB
 register	09-11-2023 14:53	Cascading Sty...	2 KB

Hình 10.11: Danh sách thư mục public\css

- Tạo tệp app.js

Cuối cùng, hãy tạo tệp ứng dụng Node.js chính, app.js. Tệp này đóng vai trò là điểm vào chính trong ứng dụng thư viện. Trong tệp này, logic cốt lõi của ứng dụng được xử lý. Các tuyến đường và phần mềm trung gian cũng được định nghĩa.

Hãy xem xét mã trong Đoạn mã 17 cho tệp app.js.

Đoạn mã 17:

```
const express = require("express"); const
mongoose = require("mongoose"); const passport =
require("passport"); const bodyParser =
require("body-parser"); const LocalStrategy =
require("passport-local"); const passportLocalMongoose =
require("passport-local-mongoose"); const User = require("../model/User");
const Book =
require("../model/Book"); const app = express();
mongoose.connect(`mongodb+srv://

lindalarrissa91:linda91@cluster0.gktucwf.mongo db.net/Library?
retryWrites=true&w=majority`); app.set("view engine",

"ejs");
app.use(bodyParser.urlencoded({ extended: true }));

app.use( require("express-session")
({ secret: "Rio là một con
chó", lưu lại:
false, saveUninitialized:

false, }) ); app.use(passport.initialize());
app.use(passport.session());
app.use(express.static(__dirname + "/public"));
```



```
// Thêm chiến lược quản trị cục bộ passport.use(

  "admin-local", new
  LocalStrategy(function (username, password, done) { if (username === "Admin" && password ===
  "12345") { return done(null, { username: "Aptech" }); } return done(null, false, {message:
  "Tên người dùng hoặc mật khẩu quản trị viên không đúng" });

});

}); passport.serializeUser(hàm (người dùng, xong) {
  // Tại đây, bạn có thể muốn tuần tự hóa dữ liệu người dùng nếu cần (ví dụ: user.id) done(null,
  user); });

passport.deserializeUser(hàm (người dùng, xong) {
  // Triển khai logic khử tuần tự hóa tại đây nếu cần done(null, user); }); // Hiển thị
  trang chủ app.get("/",

function (req, res) {

  res.render("home"); }); // Hiển
  thị
  biểu mẫu đăng ký app.get("/register",
  function (req, res) {
    res.render("đăng ký"); });

// Xử lý đăng ký người dùng app.post("/
register", async (req, res) => { const user = await User.create({ username:
  req.body.username, password: req.body.password, });
  res.redirect("/"); });

// Hiển thị biểu mẫu đăng nhập
app.get("/login", function (req, res) {
  res.render("đăng nhập"); });

// Xử lý đăng nhập người dùng app.post("/
login", hàm bất đồng bộ (req, res) {
  thử
  { const user = await User.findOne({ tên người dùng:
  req.body.username });
  nếu (người dùng)
  { const result = req.body.password === user.password;
```

```
if (result) { const
books = await Book.find({}); res.render("booklist", { books:
books }); } else { res.render("error", { errorMessage: "Mật khẩu không
khớp" }); } }
else { res.render("error", { errorMessage: "Người dùng không tồn tại" }); } } catch (error)

{ res.render("error", { errorMessage: "Đã xảy ra lỗi" }); } });

// Tuyển đăng nhập quản trị viên
app.get("/admin", function (req, res) {
  res.render("admin-login"); }); // Biểu mẫu
đăng
nhập quản trị viên app.post( "/"
admin-login",
  passport.authenticate("admin-
local", { successRedirect: "/admin-dashboard", failureRedirect: "/"
admin-error", }) ); // Tuyển lỗi quản trị app.get("/admin-
error", function (req, res) { res.render("admin-error",

{ errorMessage: "Quản trị viên
không chính xác

tên người dùng hoặc mật khẩu" }); }); //

Tuyển đường bảng điều khiển quản trị
app.get("/admin-dashboard", function (req, res) {
  if (req.isAuthenticated()) { res.render("admin-
dashboard"); } else { res.redirect("/admin"); } }); //
Lấy thông
tin chi tiết về sách từ biểu mẫu

app.post("/admin-dashboard/add-book", function (req, res)
{ if (req.isAuthenticated()) { const bookDetails = { Book_id: req.body.Book_id, Book_name:
req.body.Book_name, Author_name:
req.body.Author_name, Price:
req.body.Price, Age_group: req.body.Age_group,
Book_type: req.body.Book_type, };
```

```

// Tạo một cuốn sách mới trong bộ sưu tập "sách"
Book.create(bookDetails)
  .then((sách mới) => {
    console.log("Đã thêm sách thành công:", newBook);
    res.redirect("/admin-dashboard"); })

  .catch((lỗi) => {
    console.error("Không thêm được sách:", err);
    res.status(500).json({ lỗi: "Không thêm được sách" });
  });
} khác {
  res.redirect("/admin");
}
});
// Xử lý đăng xuất người dùng
app.get("/đăng xuất", hàm (yêu cầu, độ phân giải) {
  req.logout(hàm (lỗi) {
    nếu (sai) {
      trả về next(err);
    }
    res.redirect("/");
  });
});
hàm isLoggedIn(req, res, next) {
  nếu (req.isAuthenticated()) trả về next();
  res.redirect("/đăng nhập");
}
const port = process.env.PORT || 3000;
app.listen(cổng, hàm () {
  console.log("Máy chủ đã khởi động!");
});

```

Lưu mã với tên tệp là app.js trong thư mục library_application.

Các thao tác được trình bày trong tệp app.js như sau:

- o Nhập các gói: Nhiều gói Node.js như express, mongoose, passport, body-parser, passport-local và passport-local-mongoose được nhập. o Nhập lược đồ: Hai mô hình dữ liệu là User và Book từ thư mục mô hình cục bộ được nhập.

- o Tạo ứng dụng express: Câu lệnh const app = express()

tạo ứng dụng Express. o Kết nối cơ sở dữ

liệu: Câu lệnh mongoose.connect(`mongodb+srv://lindalarrissa91:linda91@cluster0.gktucwf.m

ongodb.net/Library?retryWrites=true&w=majority`) kết nối ứng dụng với cơ sở dữ liệu MongoDB, Library, bằng cách sử dụng chuỗi kết nối.

- o Thiết lập công cụ xem: Câu lệnh app.set("view engine", "ejs") được sử dụng để thiết lập công cụ xem thành EJS.

- o Thiết lập phần mềm trung gian: Các gói như `body-parser`, `express-session` và `passport-authenticate` được sử dụng để thiết lập phần mềm trung gian.
- o Định nghĩa chiến lược cục bộ: Một chiến lược cục bộ được định nghĩa để xác thực quản trị viên. Nếu tên người dùng `Admin` và mật khẩu `12345` khớp nhau, hàm sẽ trả về một đối tượng quản trị viên. Nếu không, nó sẽ trả về thông báo lỗi.
- o Tuần tự hóa và hủy tuần tự hóa các hàm người dùng: Các hàm được định nghĩa để tuần tự hóa và hủy tuần tự hóa các đối tượng người dùng. Các hàm này được sử dụng để quản lý các phiên người dùng.
- o Định nghĩa các tuyến đường: Ứng dụng định nghĩa một số tuyến đường để xử lý HTTP yêu cầu. Các tuyến đường là:
 - o `/`: Đây là đường dẫn đến trang chủ.
 - o `/register`: Dùng để đăng ký thành viên mới.
 - o `/login`: Dùng để xác thực thành viên hiện tại.
 - o `/admin`: Đây là tuyến đường đăng nhập của quản trị viên.
 - o `/admin-login`: Đây là nơi xử lý tên người dùng quản trị viên và mật khẩu.
 - o `/admin-error`: Hiển thị lỗi đăng nhập của quản trị viên.
 - o `/admin-dashboard`: Đây là tuyến đường đến bảng điều khiển của quản trị viên.
 - o `/admin-dashboard/add-book`: Đây là nơi xử lý việc thêm một cuốn sách mới của người quản lý.
 - o `/logout`: Đây là tuyến đường đăng xuất của thành viên/quản trị viên.
- o Xử lý đăng ký thành viên mới: Trên tuyến đường `/register`, việc đăng ký thành viên mới được xử lý bằng tên người dùng và mật khẩu được cung cấp. Tên người dùng và mật khẩu được thêm vào người dùng bộ sưu tập trong cơ sở dữ liệu Thư viện.
- o Xử lý đăng nhập thành viên hiện có: Trên tuyến đường `/login`, thông tin đăng nhập thành viên hiện có được xử lý. Tuyến đường này kiểm tra xem thành viên có tồn tại không và mật khẩu có khớp không. Nếu thành công, tuyến đường sẽ lấy danh sách sách từ cơ sở dữ liệu và hiển thị danh sách sách đó từ thư mục `views`. Nếu không, tuyến đường sẽ hiển thị thông báo lỗi.
- o Xử lý đăng nhập quản trị viên và bảng điều khiển: Đăng nhập quản trị viên được xử lý bằng một chiến lược riêng biệt, sử dụng tên người dùng và mật khẩu quản trị viên. Nếu xác thực thành công, quản trị viên sẽ được chuyển hướng đến bảng điều khiển quản trị. Nếu không, thông báo lỗi sẽ hiển thị.
- o Quản lý bảng điều khiển quản trị và thêm sách: Bảng điều khiển quản trị Tuyến đường được bảo vệ bằng xác thực. Quản trị viên có thể thêm sách mới bằng cách gửi biểu mẫu. Những cuốn sách này sẽ được thêm vào bộ sưu tập sách trong cơ sở dữ liệu Thư viện và quản trị viên sẽ được chuyển hướng đến bảng điều khiển.
- o Quản lý quy trình đăng xuất của thành viên/quản trị viên: Thành viên có thể đăng xuất bằng cách truy cập vào tuyến đường `/logout`, tuyến đường này sẽ xóa phiên làm việc và chuyển hướng họ đến trang chủ.

- o Lắng nghe yêu cầu: Ứng dụng lắng nghe trên một cổng được chỉ định, từ biến môi trường hoặc cổng 3000. Một thông báo sẽ được ghi lại khi máy chủ khởi động.

Như vậy là hoàn tất việc mã hóa cho ứng dụng.

10.4 Tải ứng dụng thư viện lên kho lưu trữ GitHub

Ứng dụng đã phát triển phải được tải lên Internet để có thể truy cập toàn cầu. Kho lưu trữ GitHub được sử dụng cho mục đích này. Kho lưu trữ này có thể lưu trữ mã, tệp, lịch sử phiên bản, v.v. của các ứng dụng. Để tải ứng dụng thư viện lên kho lưu trữ GitHub, hãy tạo một không gian lưu trữ độc quyền trên GitHub.

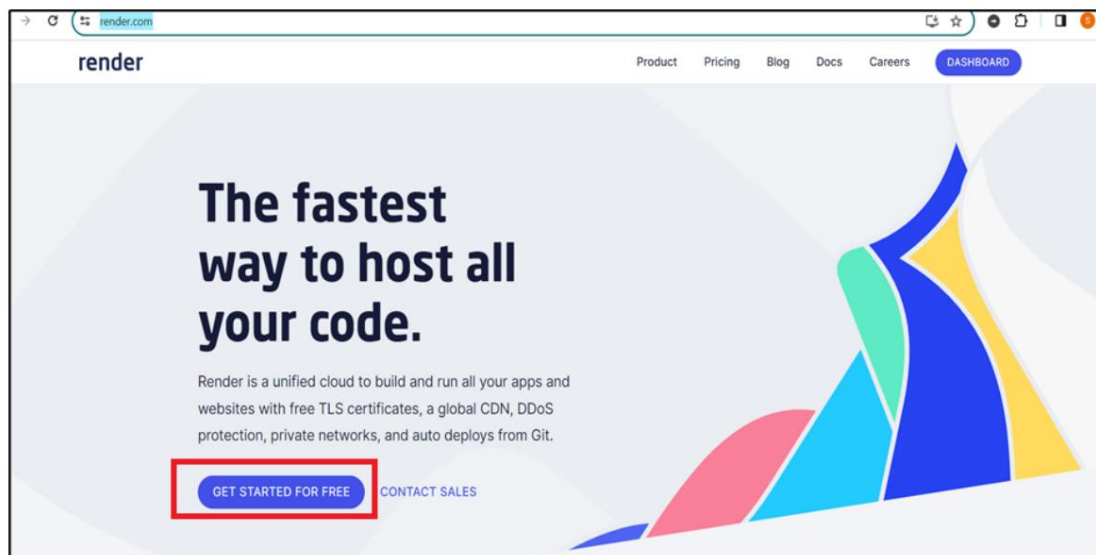
Tham khảo Phụ lục B để biết mô tả chi tiết về cách:

- Tạo kho lưu trữ trong tài khoản GitHub
- Tải các tài nguyên ứng dụng lên kho lưu trữ

10.5 Triển khai ứng dụng thư viện trên Render từ kho lưu trữ GitHub

Render là một nền tảng điện toán đám mây giúp các nhà phát triển triển khai ứng dụng của họ trên đám mây. Ứng dụng thư viện được tải lên kho lưu trữ GitHub. Để triển khai ứng dụng trên Render từ kho lưu trữ GitHub, hãy thực hiện các bước sau:

1. Để truy cập nền tảng Render, trong trình duyệt, hãy điều hướng đến URL <https://render.com/>, và nhấp vào **BẮT ĐẦU MIỄN PHÍ** như thể hiện trong Hình 10.12.



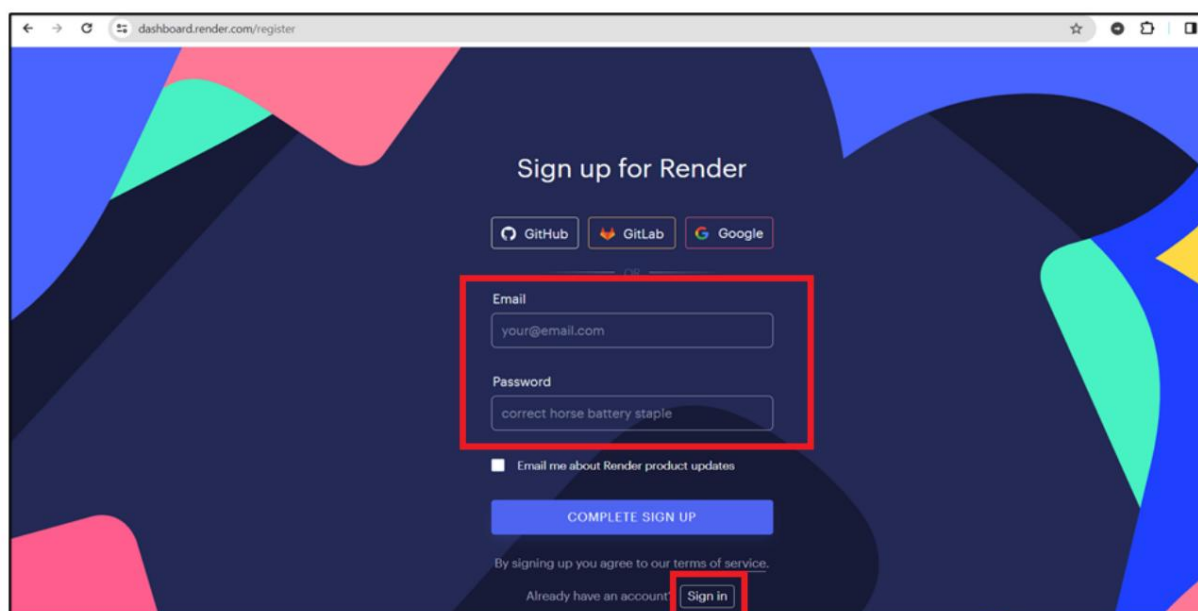
Hình 10.12: Trang chủ của Render

2. Để đăng nhập bằng tài khoản GitHub, trên trang Đăng ký Render , hãy nhập các thông tin chi tiết đã cho và nhấp vào Đăng nhập như thể hiện trong Hình 10.13.

Ví dụ, ở đây, tài khoản GitHub được sử dụng để đăng nhập vào Render bằng email là Linda.larrissa91@gmail.com và mật khẩu là [linda91@](#).

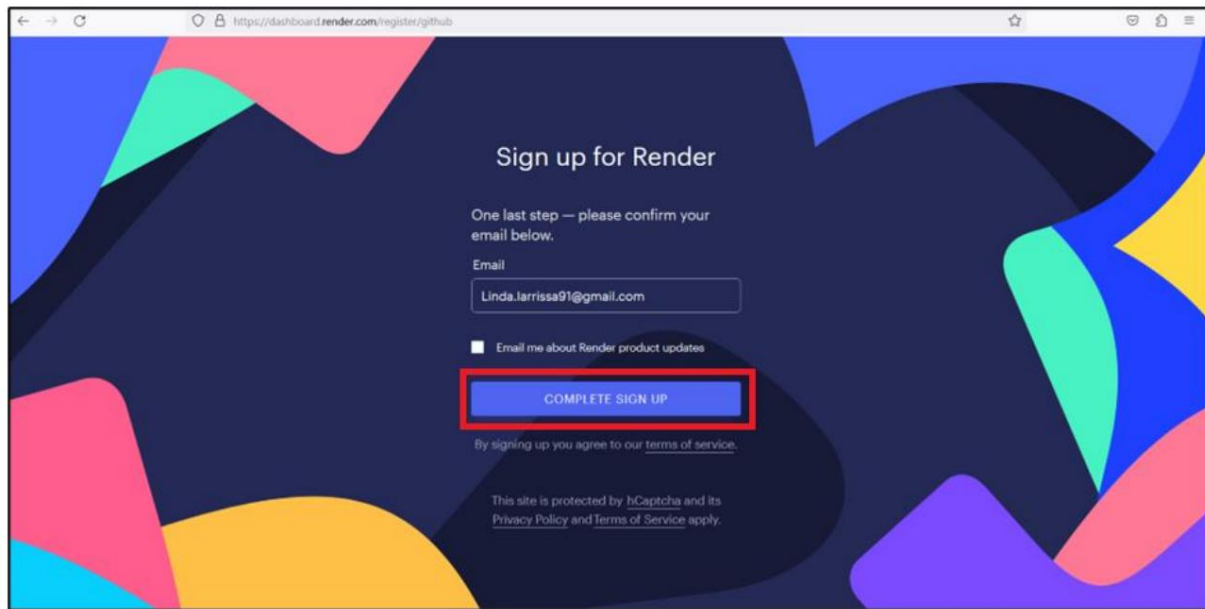
- Trong hộp Email , nhập ID tài khoản Gitbub. -
- Trong hộp Mật khẩu , nhập mật khẩu.

Nếu đã sử dụng tài khoản hiện có để đăng nhập vào Render, hãy cung cấp ID email, mật khẩu và nhấp vào Đăng nhập.



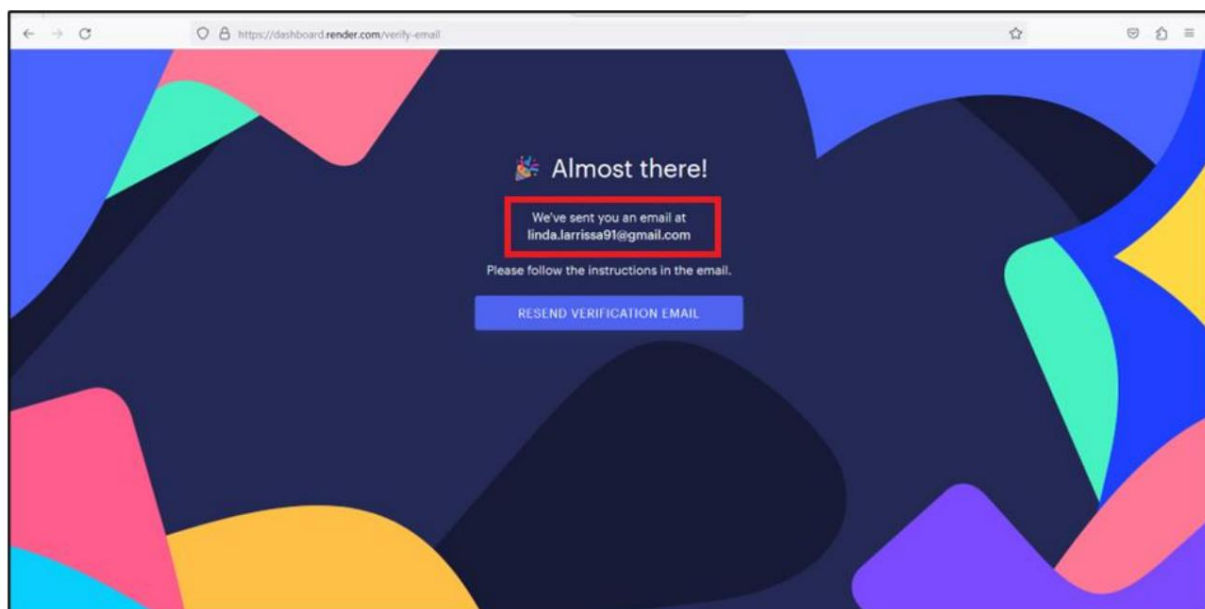
Hình 10.13: Đăng ký trang Render

3. Để hoàn tất quá trình đăng ký, trên trang Đăng ký Render , hãy nhấp vào HOÀN THÀNH ĐĂNG KÝ như thể hiện trong Hình 10.14.



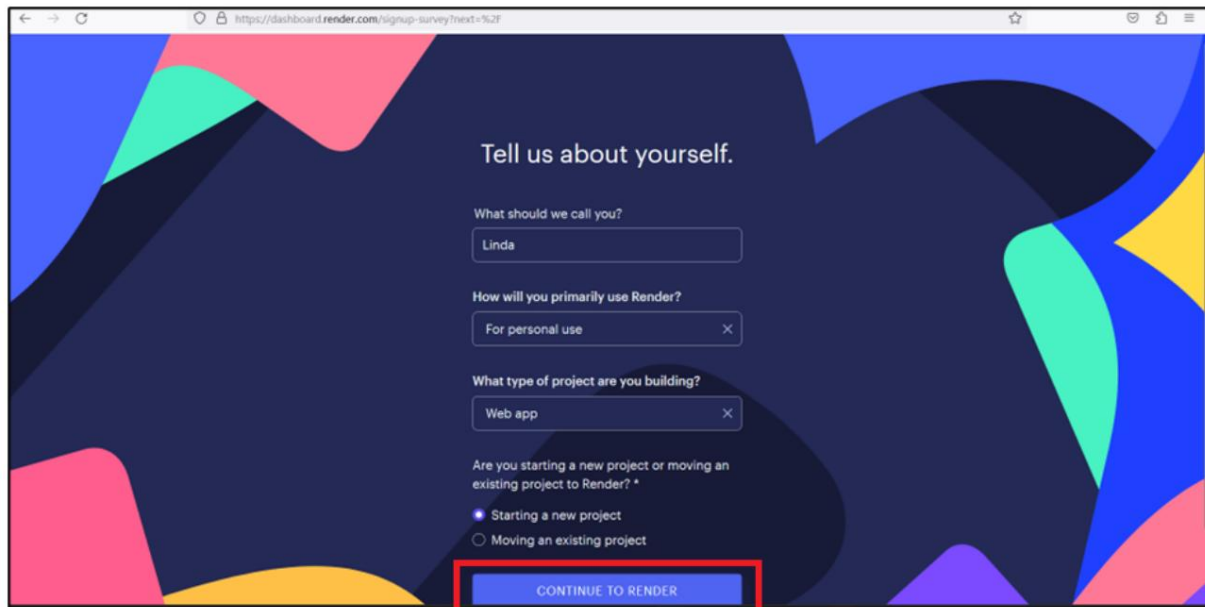
Hình 10.14: Đăng ký Trang Render - Xác nhận Email

4. Để kích hoạt tài khoản Render , hãy nhấp vào liên kết trong email theo hướng dẫn trên trang Almost there như thể hiện trong Hình 10.15.



Hình 10.15: Gần đến nơi rồi

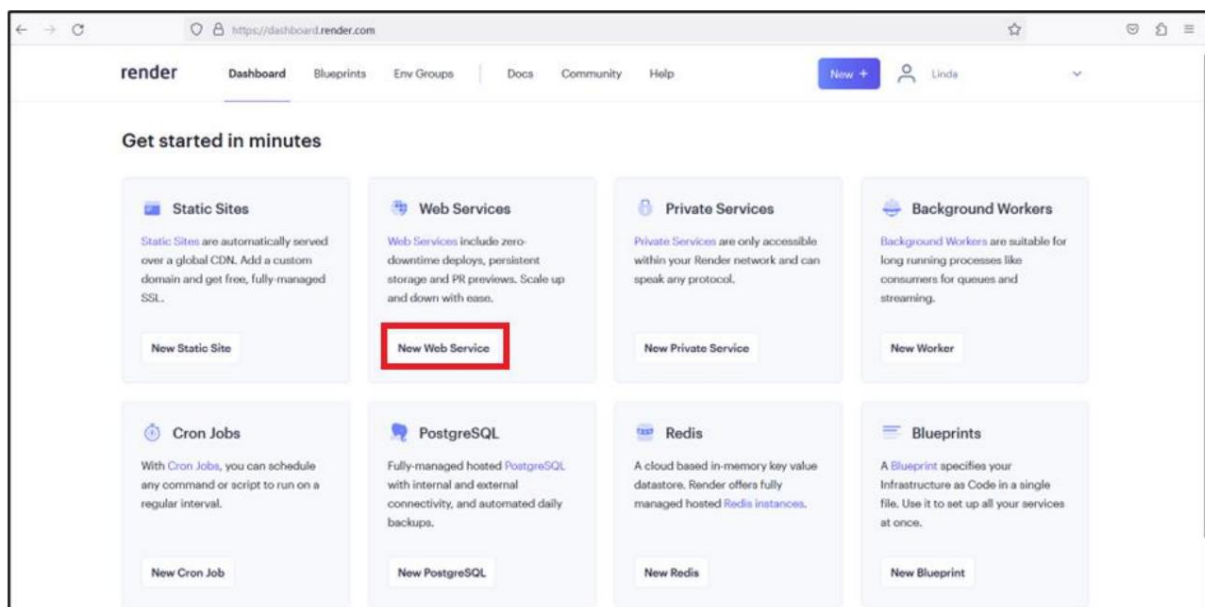
5. Để tiếp tục Render, trên trang Hãy cho chúng tôi biết về bạn , hãy điền thông tin cần thiết và nhấp vào TIẾP TỤC RENDER như minh họa trong Hình 10.16.



The screenshot shows a web browser window with the URL <https://dashboard.render.com/signup-survey?next=/52f>. The page has a dark blue background with colorful abstract shapes. The main heading is "Tell us about yourself." Below it are several form fields: "What should we call you?" with the text "Linda"; "How will you primarily use Render?" with a dropdown menu set to "For personal use"; and "What type of project are you building?" with a dropdown menu set to "Web app". There is also a question "Are you starting a new project or moving an existing project to Render? *" with two radio button options: "Starting a new project" (selected) and "Moving an existing project". At the bottom, there is a blue button labeled "CONTINUE TO RENDER" which is highlighted with a red rectangular box.

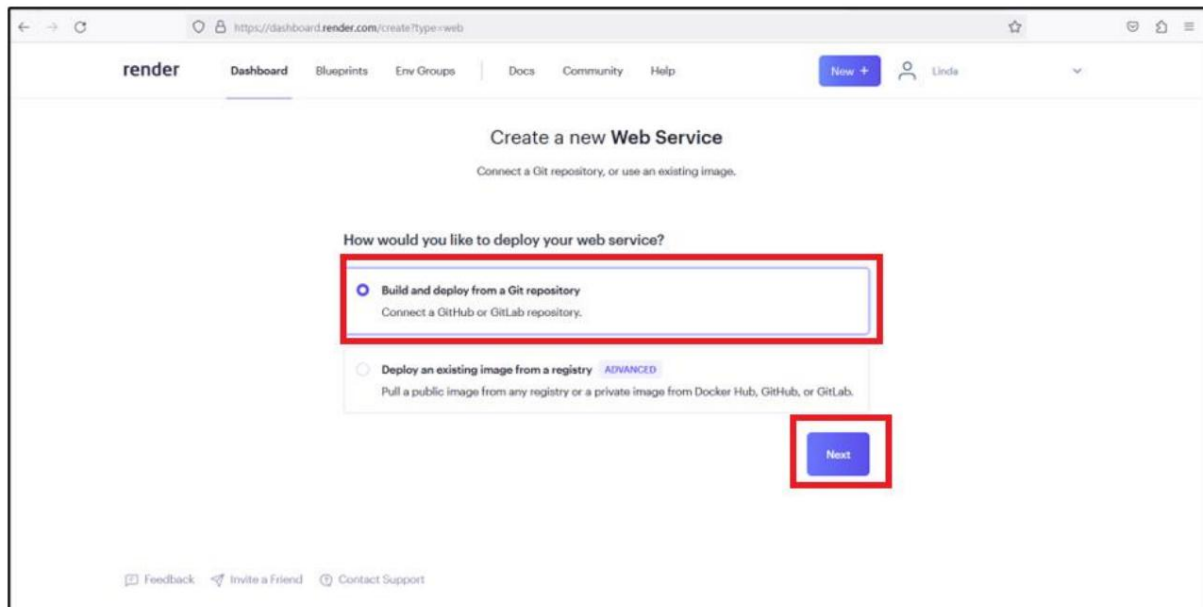
Hình 10.16: Hãy cho chúng tôi biết về bạn Trang

6. Để tạo một dịch vụ Web mới, trên trang Bảng điều khiển kết xuất , bên dưới Web Dịch vụ, nhấp vào Dịch vụ web mới như thể hiện trong Hình 10.17.



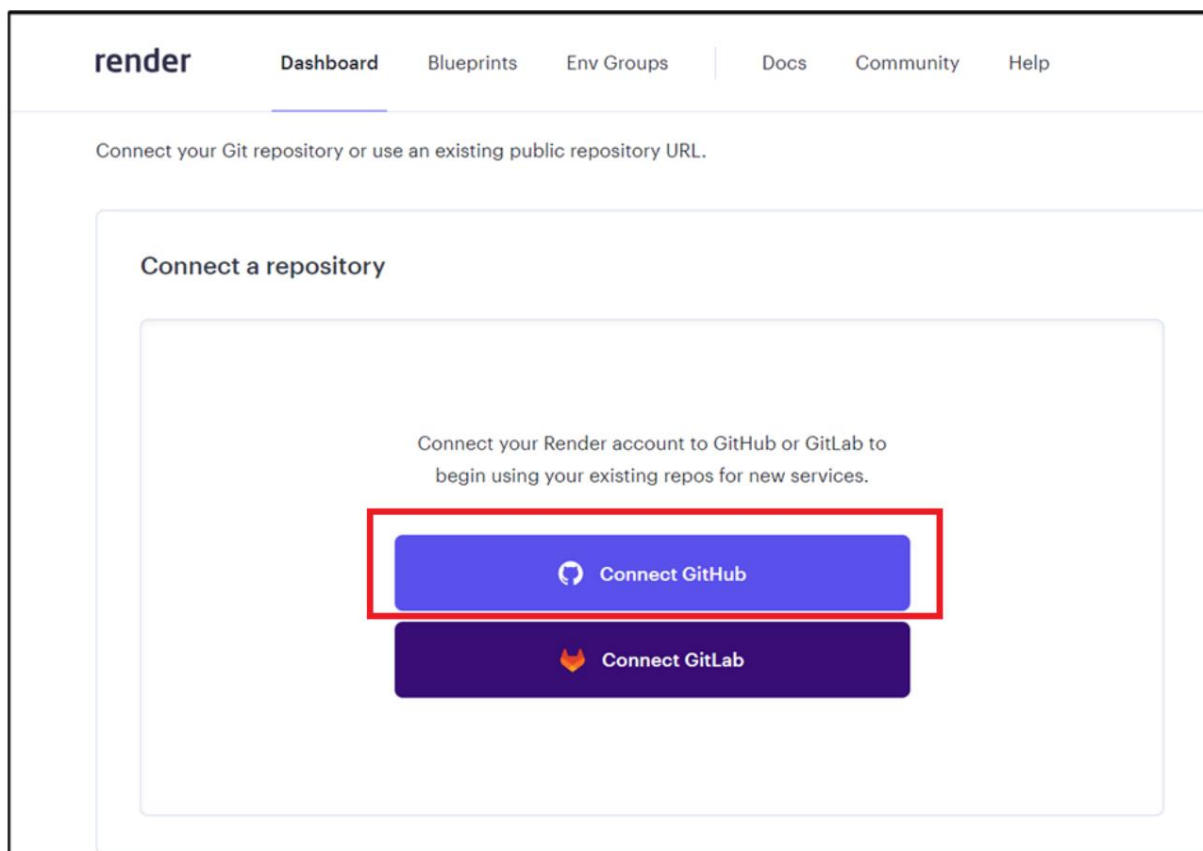
Hình 10.17: Trang bảng điều khiển kết xuất

7. Để triển khai dịch vụ Web, trên trang Tạo dịch vụ Web mới , hãy nhấp vào Xây dựng và triển khai từ kho lưu trữ Git và nhấp vào Tiếp theo như thể hiện trong Hình 10.18.



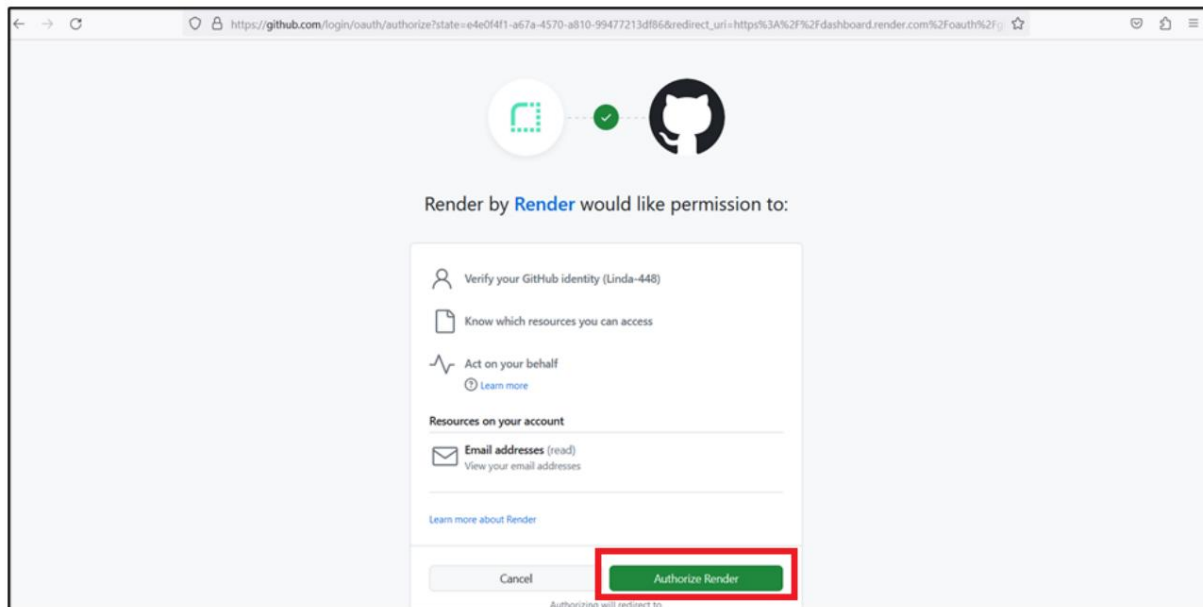
Hình 10.18: Tạo Trang Dịch vụ Web mới

- Để kết nối kho lưu trữ, trên trang Kết nối kho lưu trữ, hãy nhấp vào Kết nối GitHub như minh họa trong Hình 10.19.



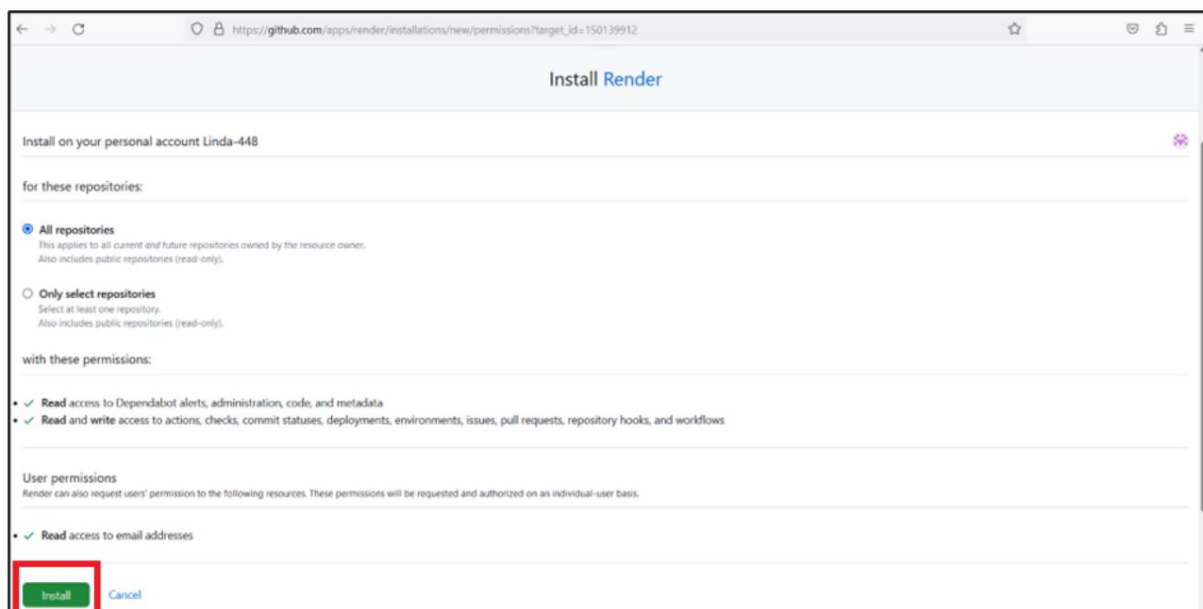
Hình 10.19: Kết nối Trang Kho lưu trữ

9. Để ủy quyền cho Render, trên Render by Render muốn được phép trang, nhấp vào Ủy quyền kết xuất như minh họa trong Hình 10.20.



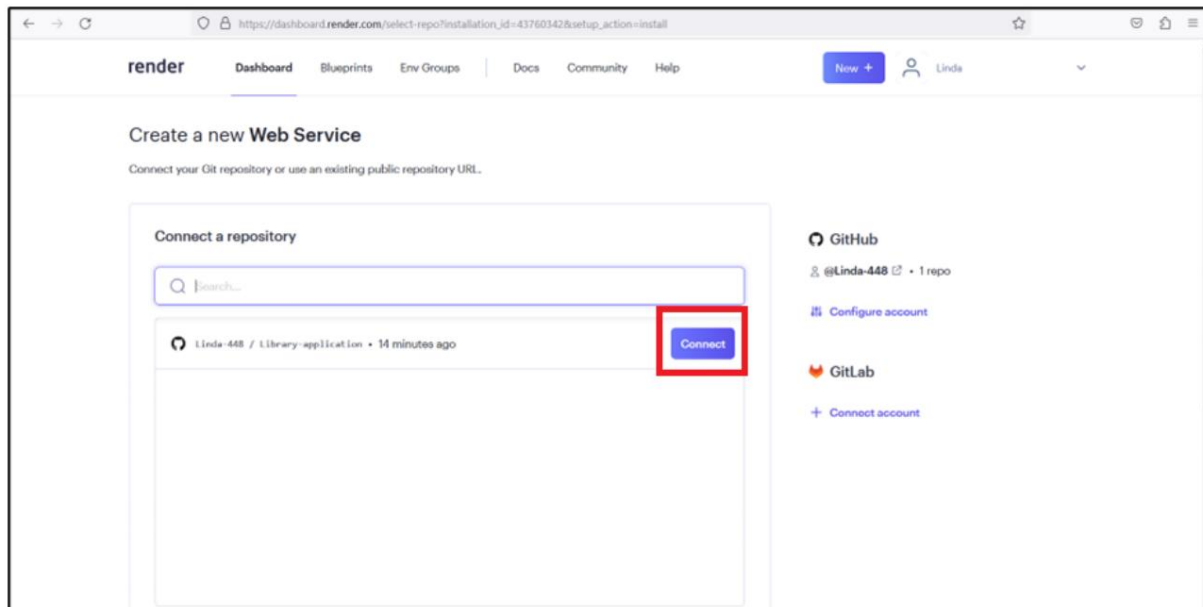
Hình 10.20: Render by Render muốn được cấp quyền truy cập vào Trang

10. Để cài đặt Render trên tài khoản GitHub, trên trang Cài đặt Render, hãy nhấp vào Cài đặt như minh họa trong Hình 10.21.



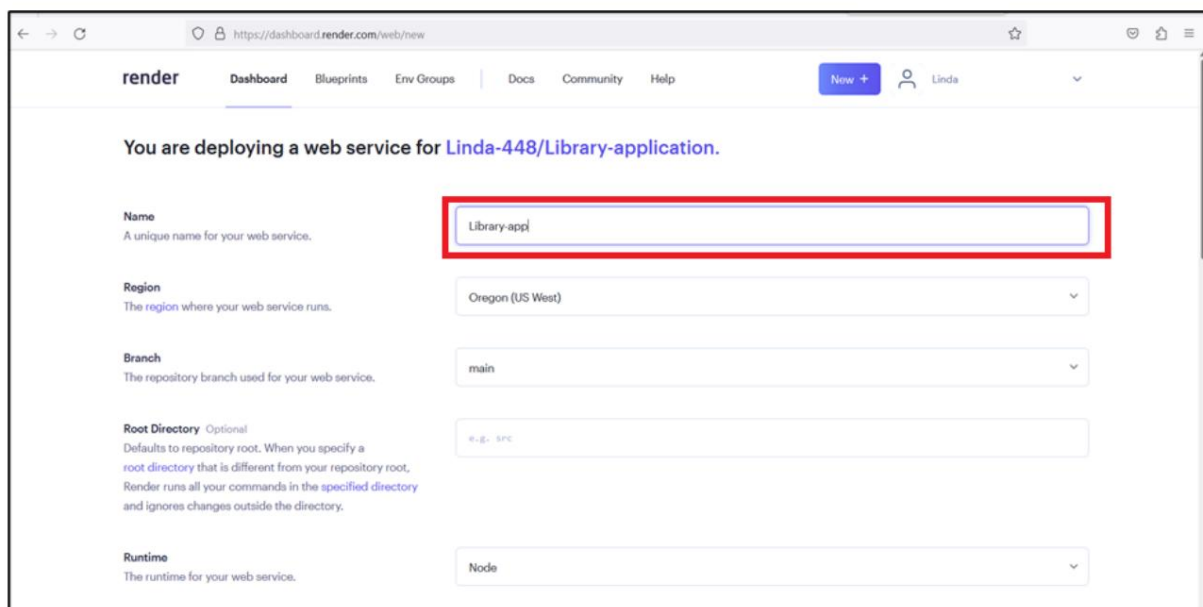
Hình 10.21: Cài đặt trang Render

11. Để kết nối kho lưu trữ, trên trang Tạo dịch vụ web mới, hãy nhấp vào Kết nối như minh họa trong Hình 10.22.



Hình 10.22: Kết nối kho lưu trữ

12. Để cung cấp tên duy nhất cho dịch vụ Web, trên trang Render Dashboard, trong hộp Tên, hãy nhập Library-app như minh họa trong Hình 10.23.



Hình 10.23: Tên dịch vụ web

13. Để cung cấp lệnh bắt đầu, trên trang Render Dashboard, cuộn xuống và trong hộp Start Command, nhập node app.js như minh họa trong Hình 10.24.

render Dashboard Blueprints Env Groups Docs Community Help

Runtime: The runtime for your web service.

Build Command: This command runs in the root directory of your repository when a new version of your code is pushed, or when you deploy manually. It is typically a script that installs libraries, runs migrations, or compiles resources needed by your app.

Start Command: This command runs in the root directory of your app and is responsible for starting its processes. It is typically used to start a webserver for your app. It can access environment variables defined by you in Render.

Please enter your payment information to select an instance type with higher limits.

Instance Type	RAM	CPU	Price
☒ Free	512 MB	0.1 CPU	\$0 / month

Hình 10.24: Lệnh khởi động cho dịch vụ web

14. Để tạo dịch vụ Web, trên trang Render Dashboard, cuộn xuống và nhấp vào Create Web Service như minh họa trong Hình 10.25.

render Dashboard Blueprints Env Groups Docs Community Help

Pro 4 GB 2 CPU \$85 / month

Pro Plus 8 GB 4 CPU \$175 / month

Pro Max 16 GB 4 CPU \$225 / month

Pro Ultra 32 GB 8 CPU \$450 / month

Need a custom instance type? We support up to 512 GB RAM and 64 CPUs.

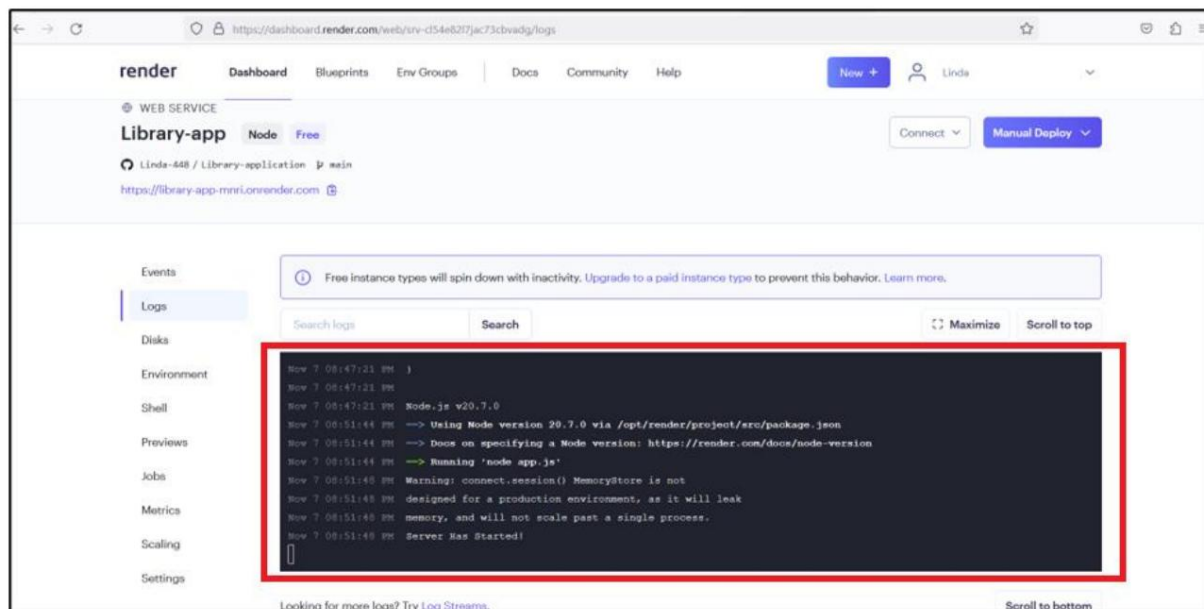
Unlike paid services, free services scale down when inactive. Learn more about free instance type limits.

Advanced

Create Web Service

Hình 10.25: Tạo dịch vụ web trên trang bảng điều khiển kết xuất

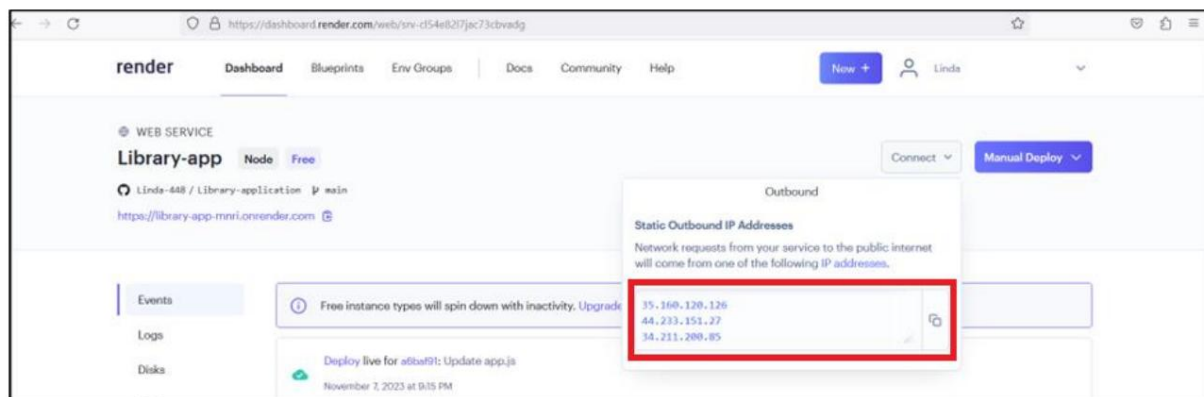
15. Lưu ý rằng ứng dụng Thư viện đã được triển khai và máy chủ đã khởi động như được hiển thị trong Hình 10.26.



Hình 10.26: Triển khai ứng dụng

16. Để xác định địa chỉ IP của Render cho ứng dụng Thư viện, trên trang Render Dashboard, hãy nhấp vào danh sách thả xuống Connect như minh họa trong Hình 10.27.

Các địa chỉ IP tĩnh gửi đi sẽ được hiển thị. Không an toàn khi sử dụng địa chỉ IP 0.0.0.0/0, được chỉ định khi kết nối ứng dụng với cơ sở dữ liệu MongoDB. Do đó, hãy sử dụng bất kỳ địa chỉ IP nào trong danh sách.

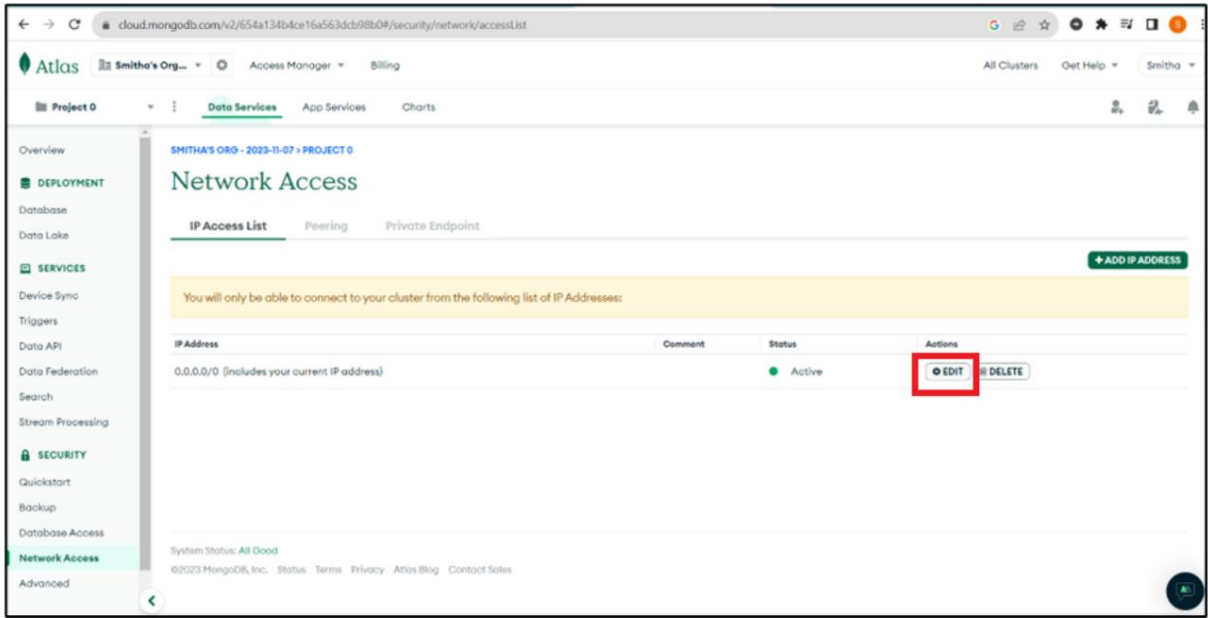


Hình 10.27: Địa chỉ IP trên Trang bảng điều khiển kết xuất

17. Để cập nhật địa chỉ IP, hãy điều hướng đến tài khoản đám mây MongoDB. Trang Triển khai cơ sở dữ liệu đang hoạt động.

Ở bên trái, nhấp vào Truy cập mạng.

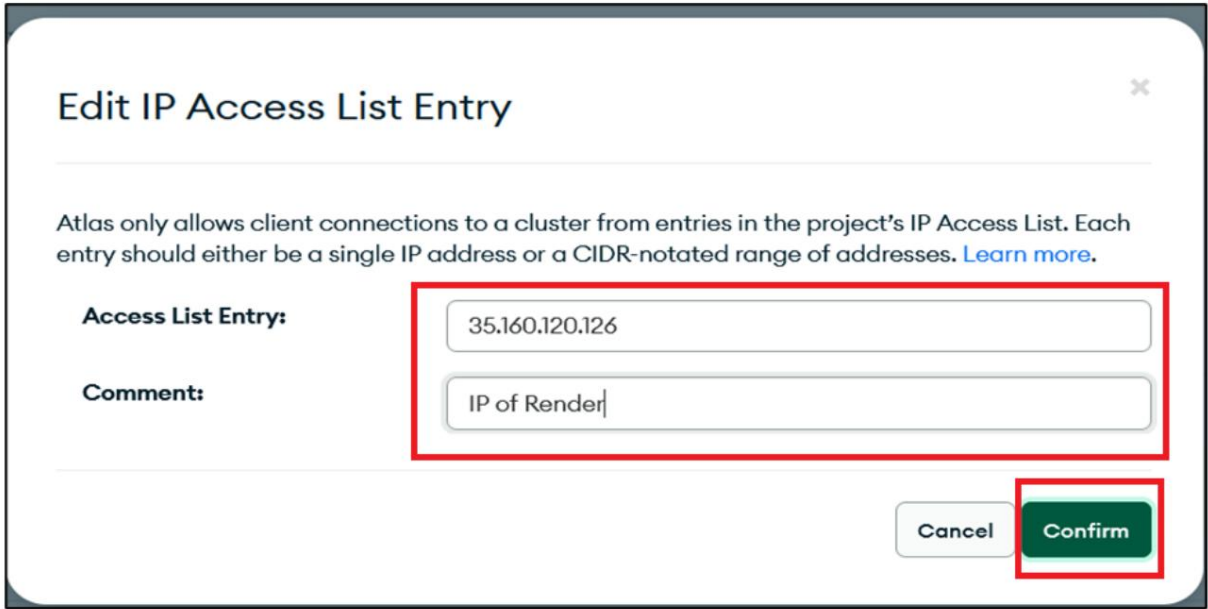
Trên trang Truy cập mạng, nhấp vào Chính sửa như thể hiện trong Hình 10.28.



Hình 10.28: Trang truy cập mạng

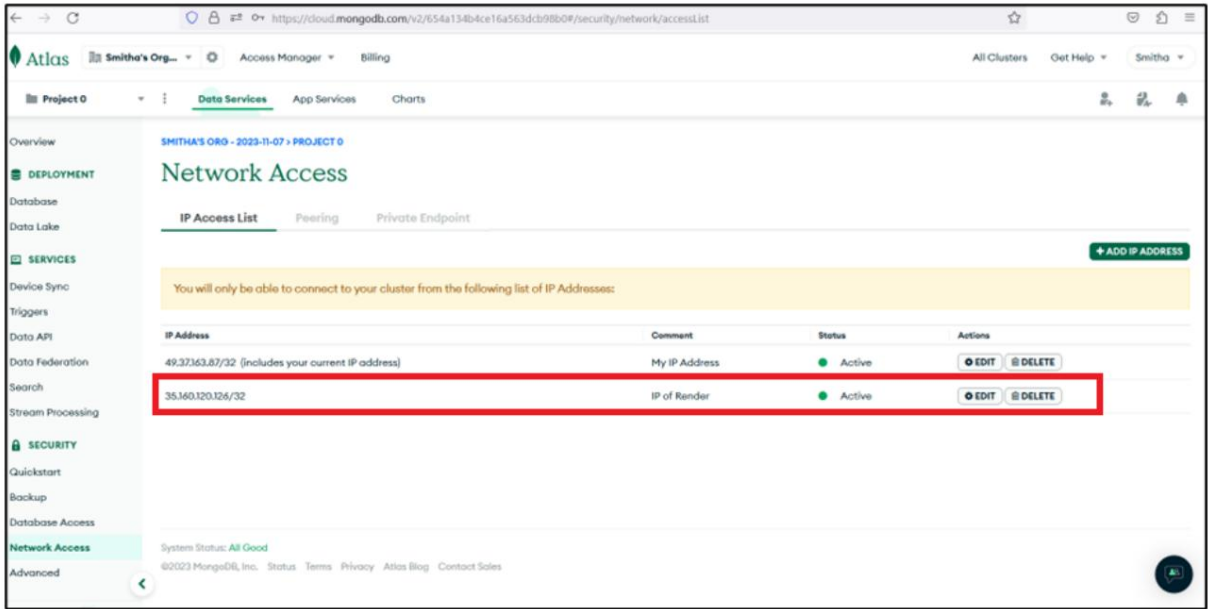
18. Để chỉnh sửa địa chỉ IP từ 0.0.0.0/0 đến 35.160.120.126, trên trang Chỉnh sửa mục nhập danh sách truy cập IP , trong hộp mục nhập danh sách truy cập , hãy nhập 35.160.120.126.

Để chỉ định một chú thích, trong hộp Chú thích , nhập IP của Render và nhấp vào Xác nhận như minh họa trong Hình 10.29.



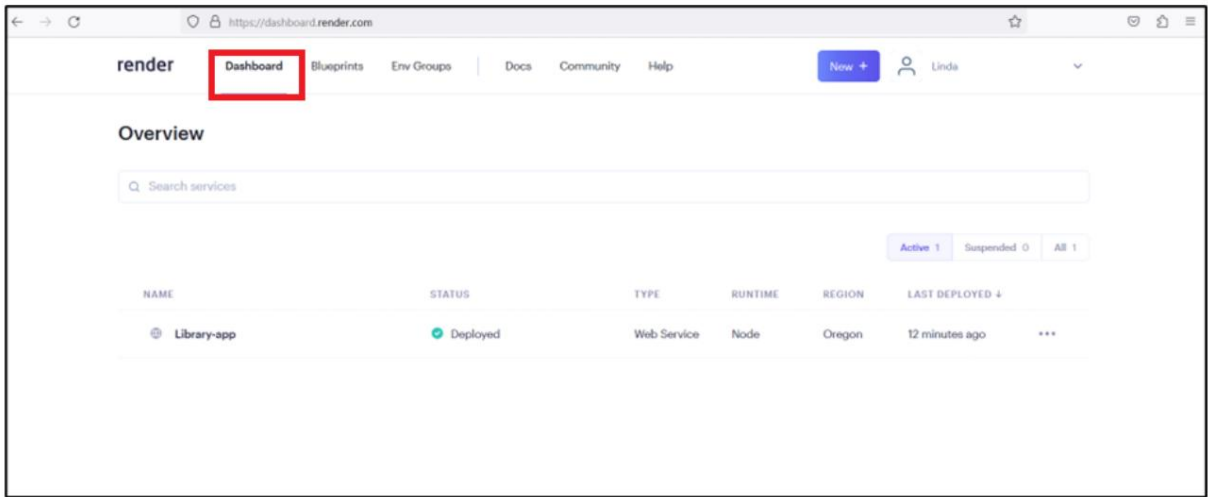
Hình 10.29: Trang chỉnh sửa mục nhập danh sách truy cập IP

19. Địa chỉ IP được cập nhật như thể hiện trong Hình 10.30.



Hình 10.30: Địa chỉ IP được cập nhật trên trang Truy cập mạng

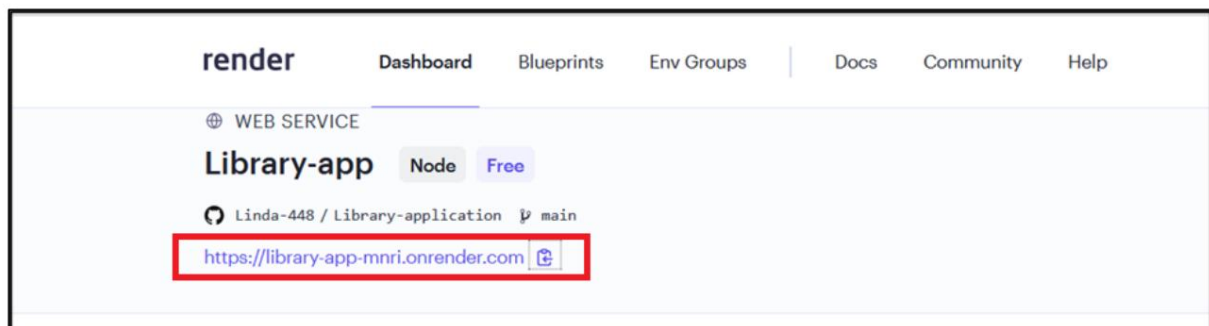
20. Để xem trạng thái của dự án ứng dụng Thư viện , hãy điều hướng đến Render và nhấp vào Bảng điều khiển như thể hiện trong Hình 10.31.



Hình 10.31: Trạng thái dự án trên trang bảng điều khiển kết xuất

21. Để sao chép URL, trên trang Render Dashboard , hãy nhấp vào Library-app, chọn URL và nhấn Ctrl + C như thể hiện trong Hình 10.32.

URL, <https://library-app-mnri.onrender.com>, được sao chép vào bảng tạm.



Hình 10.32: URL trên Trang bảng điều khiển kết xuất

10.6 Truy cập ứng dụng thư viện trong trình duyệt web

Ứng dụng thư viện hiện đã được triển khai và sẵn sàng để truy cập trong trình duyệt web bằng URL.

Thực hiện các bước sau:

1. Mở trình duyệt web và dán URL đã sao chép. Trang chủ của ứng dụng thư viện sẽ được hiển thị.

Để tạo thành viên mới, hãy nhấp vào Đăng ký người dùng thành viên như thể hiện trong Hình 10.33.

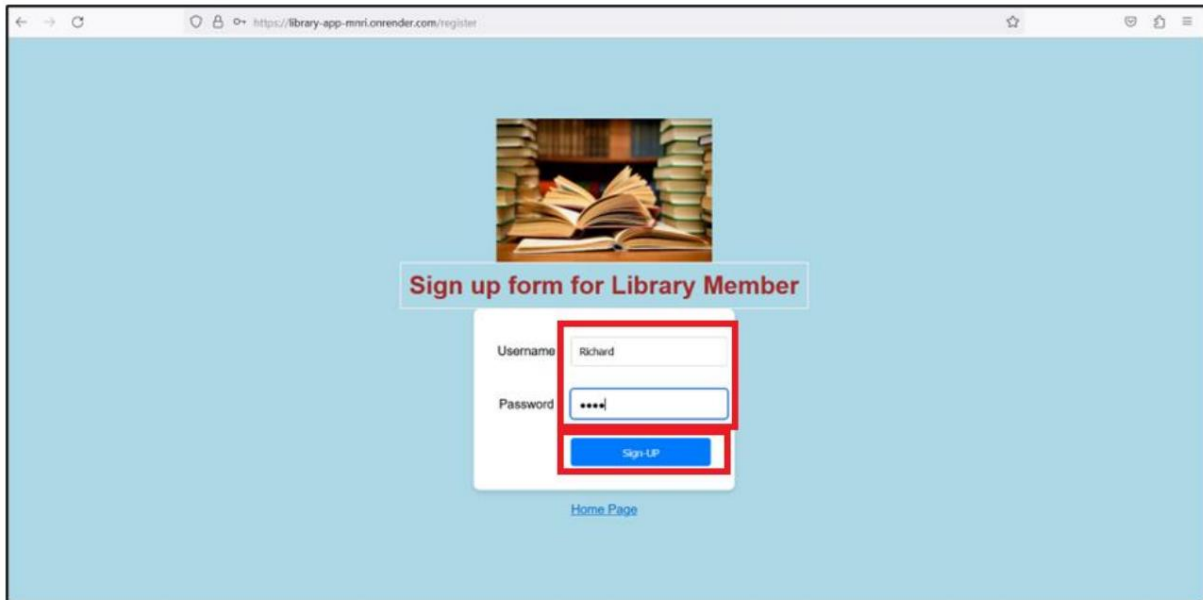


Hình 10.33: Trang chủ của ứng dụng Thư viện

2. Mẫu đăng ký thành viên thư viện mới sẽ được hiển thị.

Để cung cấp tên người dùng, trong hộp Tên người dùng, hãy nhập Richard.

Để cung cấp mật khẩu, trong hộp Mật khẩu, nhập 1234 và nhấp vào Đăng ký như minh họa trong Hình 10.34.



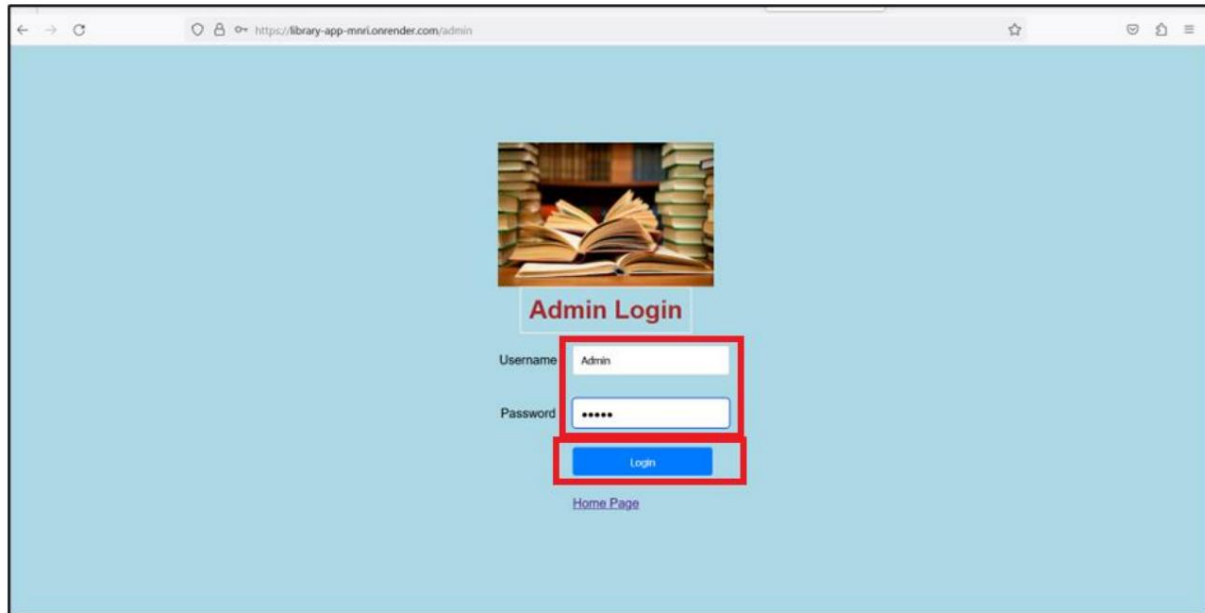
Hình 10.34: Mẫu đăng ký thành viên thư viện

- Để đăng nhập với tư cách quản trị viên, trên trang chủ, hãy nhấp vào Đăng nhập quản trị viên như thể hiện trong Hình 10.35.



Hình 10.35: Trang chủ của ứng dụng Thư viện

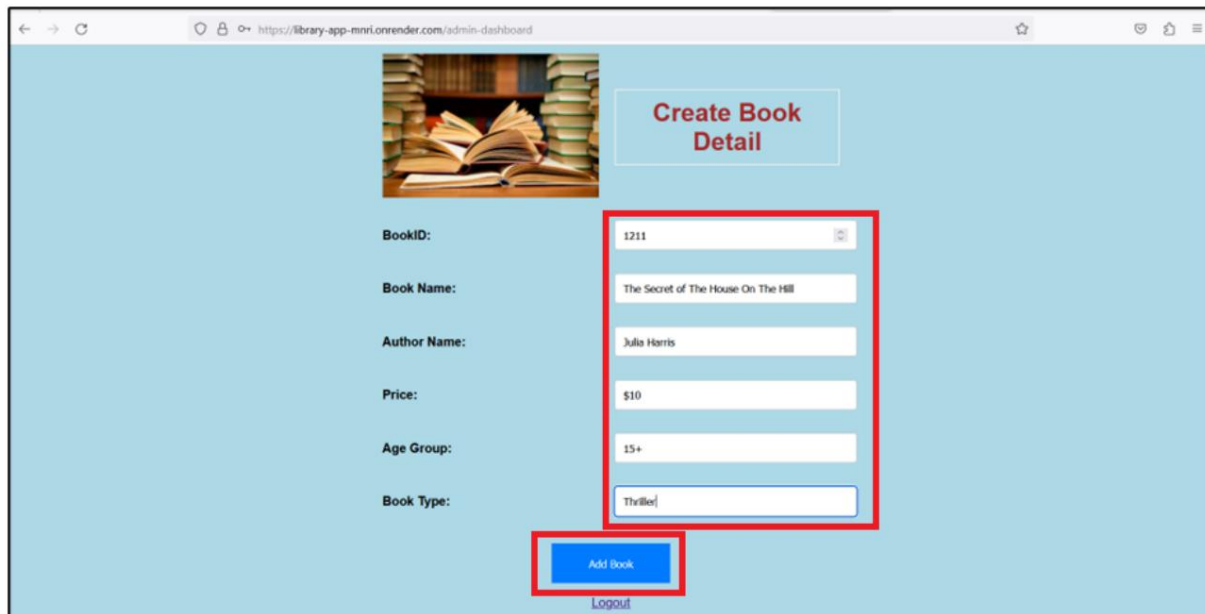
- Để cung cấp tên người dùng quản trị viên, trên trang Đăng nhập quản trị viên, trong Hộp tên người dùng, nhập Admin.
Để cung cấp mật khẩu, trong hộp Mật khẩu, nhập 12345 và nhấp vào Đăng nhập như minh họa trong Hình 10.36.



Hình 10.36: Trang đăng nhập quản trị của ứng dụng thư viện

5. Để thêm chi tiết sách mới, trên trang Tạo chi tiết sách , hãy nhập cung cấp thông tin chi tiết và nhấp vào Thêm sách như thể hiện trong Hình 10.37. Trong ô BookID , nhập 1211. Trong ô Tên sách , nhập Bí mật của ngôi nhà trên đồi. Trong ô Tên tác giả , hãy nhập Julia Harris. Trong ô Giá , nhập 10 đô la. Trong ô Nhóm tuổi , nhập 15+. Trong hộp Loại sách , nhập Thriller. Thông tin chi tiết về cuốn sách sẽ được thêm vào cơ sở dữ liệu.

Thêm thông tin chi tiết về sách nếu cần hoặc nhấp vào Đăng xuất để quay lại trang chủ.



The screenshot shows the 'Create Book Detail' form in the library application admin dashboard. The form includes the following fields:

- BookID: 1211
- Book Name: The Secret of The House On The Hill
- Author Name: Julia Harris
- Price: \$10
- Age Group: 15+
- Book Type: Thriller

A red box highlights the input fields, and another red box highlights the 'Add Book' button. The 'Logout' link is visible at the bottom.

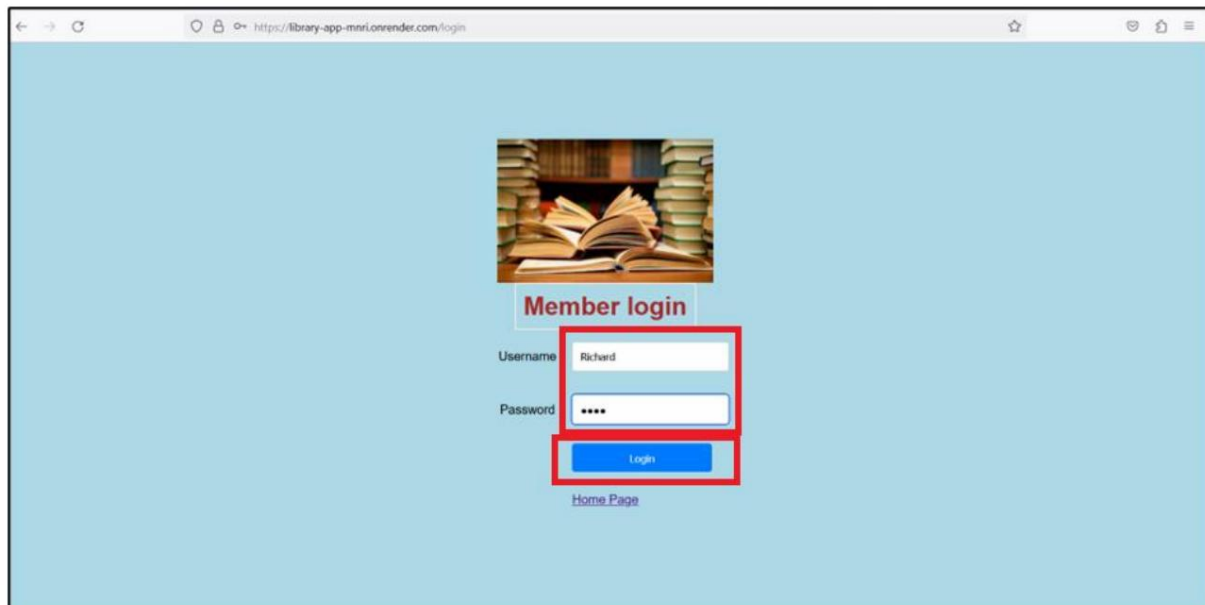
Hình 10.37: Tạo trang chi tiết sách của ứng dụng Thư viện

6. Để đăng nhập với tư cách là thành viên hiện tại, trên trang chủ Thư viện , hãy nhấp vào Đăng nhập thành viên như thể hiện trong Hình 10.38.



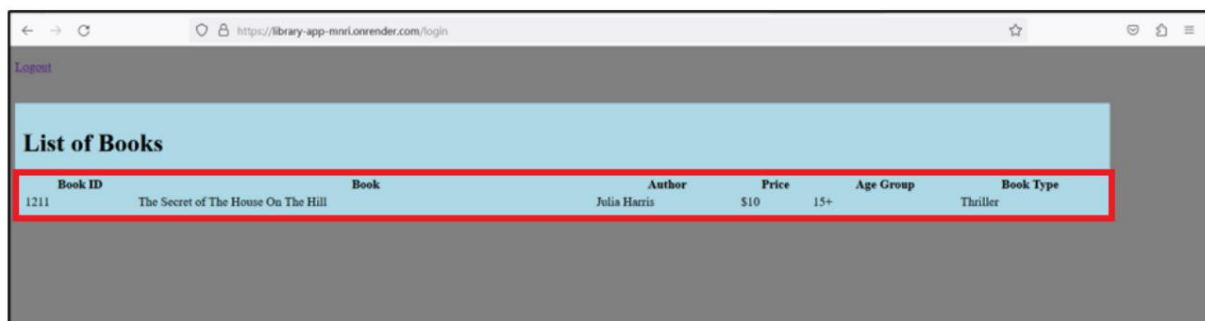
Hình 10.38: Trang chủ Thư viện

7. Để cung cấp tên người dùng hiện tại, trong hộp Tên người dùng , hãy nhập Richard.
Để cung cấp mật khẩu, trong hộp Mật khẩu , nhập 1234 và nhấp vào Đăng nhập như minh họa trong Hình 10.39.



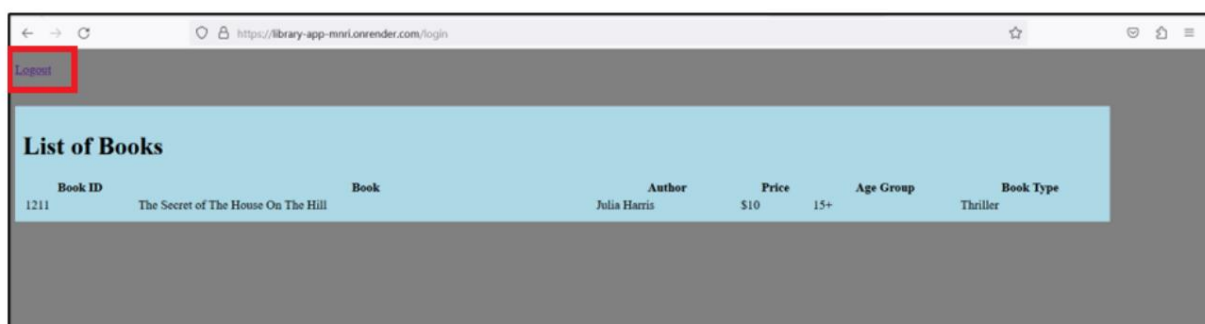
Hình 10.39: Trang đăng nhập thành viên

8. Trên trang Danh sách sách , danh sách các cuốn sách có sẵn cho thành viên được hiển thị như trong Hình 10.40.



Hình 10.40: Danh sách sách trên Trang Danh sách sách

9. Để trở về trang chủ của ứng dụng thư viện, trên Danh sách Sách trang, nhấp vào Đăng xuất như thể hiện trong Hình 10.41.



Hình 10.41: Trang chủ Thư viện

10.7 Tóm tắt

Chuỗi kết nối được tạo trong cơ sở dữ liệu đám mây MongoDB kết nối ứng dụng Web với đám mây MongoDB.

Các gói Node.js được cài đặt trong ứng dụng Web bằng cách sử dụng npm lệnh.

Một số nhiệm vụ quan trọng trong phát triển ứng dụng Web là:

- o Thiết lập công cụ xem và phần mềm trung gian
- o Xác định chiến lược và tuyến đường địa phương
- o Tuần tự hóa và hủy tuần tự hóa các hàm người dùng
- o Lắng nghe yêu cầu

Các lược đồ và bảng định kiểu phải được tạo trong các thư mục ứng dụng Web tương ứng. Ứng dụng Web

có thể được tải lên kho lưu trữ GitHub để cộng tác và kiểm soát phiên bản tốt hơn.

Render có thể được sử dụng để triển khai các ứng dụng Web trên đám mây.

Kiểm tra kiến thức của bạn

1. Sắp xếp các bước để tạo một ứng dụng Web đơn giản theo trình tự.

i. Tạo ứng dụng web bằng Node.js ii. Kết nối ứng dụng web với cơ sở dữ liệu iii. Tải ứng dụng lên GitHub Repository iv. Triển khai ứng dụng trên Render v. Truy cập ứng dụng trên trình duyệt web

- a) ii, i, iii, iv và
- vb) i, ii, iii, iv và
- vc) v, iv, iii, ii và
- id) i, ii, iii, v và iv

2. Bạn sẽ sử dụng lệnh nào để tạo một dự án mới trong ứng dụng? thư mục?

- a) npm init b)
- npm init -yc) npm
- init -xd) npm

3. Gói nào trong Node.js được sử dụng để phân tích cú pháp phần thân của các yêu cầu HTTP POST? a) mã hóa b) phần thân c) phân tích cú pháp d) phần thân

4. Gói nào được sử dụng để thực hiện xác thực bằng tên người dùng và mật khẩu trong các ứng dụng Node.js? a) passport-local b) local c) authentication d) ejs

5. Mục đích của lệnh npm install ejs là gì? a) Cài đặt môi trường chạy EJS b) Cài đặt gói JavaScript c) Cài đặt máy chủ d) Cài đặt thư viện EJS

Câu trả lời để kiểm tra kiến thức của bạn

1	Một
2b	
3	ngày
4	Một
5	ngày

Hãy tự mình thử nhé

1. Tạo một ứng dụng web đơn giản cho Cổng thông tin Hộ chiếu. Thực hiện đưa ra các hoạt động và triển khai ứng dụng trên Render.
 - Thêm hồ sơ mới và tham khảo các thông số như Tên, Tuổi, Tên ngân hàng và Bằng chứng ID.
 - Xóa hồ sơ.
 - Cập nhật hồ sơ.
 - a) Nhập các gói cần thiết.
 - b) Tạo một cơ sở dữ liệu có tên là Passport.
 - c) Thêm bộ sưu tập Passport_Collection vào cơ sở dữ liệu Passport và chèn 3 tài liệu như được đưa ra trong Bảng 10.2.

Tên	Tuổi	Tên ngân hàng	Bằng chứng ID
Alex	30	Bank of America	Thẻ xanh
John	21	Citigroup	Đăng ký cử tri
Harry	26	US Bancorp	Thẻ xanh

Bảng 10.2: Hồ sơ hộ chiếu

- a) Xem các tài liệu đã chèn.
- b) Đổi tên Harry thành Charlie.
- c) Xóa tài liệu có tên là Alex.
- d) Hiển thị kết quả trên trình duyệt.
- e) Triển khai ứng dụng trên Render.